DATA 260 Homework 5

Haozhe Wang

GitHub repository: https://github.com/HauZer-HaoZheWang/data260-8561

Commit used for report: 699e7d0860f9ffe6a9ae02da92ee15857ba68c2f

Submission tag: hw5

Hardware: Model Name: MacBook Pro, Chip: Apple M4, Memory: 16 GB

Local model: qwen3:8b through Ollama

Section 0 Personal Configuration

Value	Result
SID4	8561
PORT_BASE	8461
PREFIX	s8561
SEED	8561
VERIFY_SEED	268561
DOMAIN_ID	1 - Clinical Trials

Part 1 Database and API

The FastAPI and MySQL service adds Sponsor and Trial CRUD, a Sponsor to Trial relationship, unique NCT code validation, pagination, timestamps, and explicit 404, 409, and 422 responses.

SQLAlchemy relationship models

```
class Sponsor(Base):
    __tablename__ = "sponsors"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(255), nullable=False)
    contact_name = Column(String(255), nullable=False)
    email = Column(String(255), nullable=False, unique=True)
    created_at = Column(
        DateTime,
        nullable=False,
        server_default=func.now(),
    )
    updated_at = Column(
        DateTime,
        nullable=False,
        server_default=func.now(),
        onupdate=func.now(),
    )

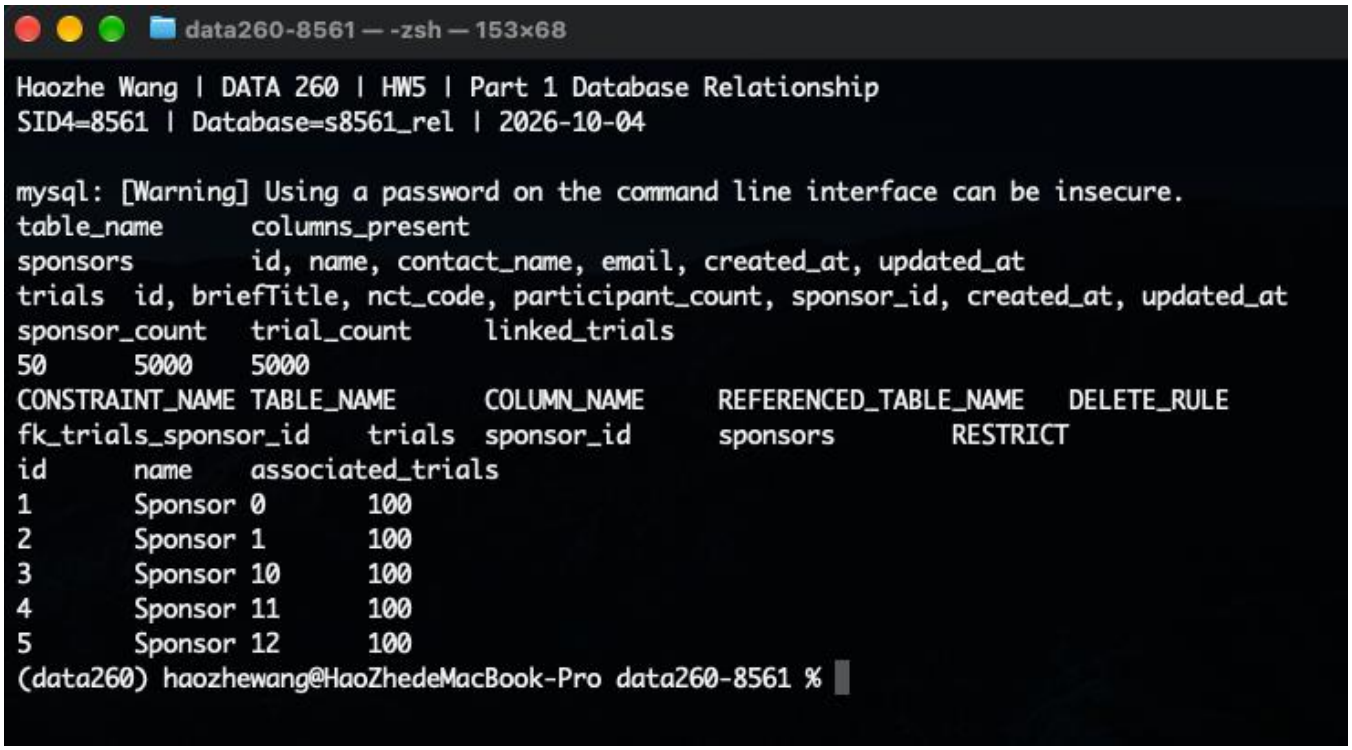
    trials = relationship(
        "Trial",
        back_populates="sponsor_entity",
        passive_deletes=True,
    )


class Trial(Base):
    __tablename__ = "trials"

    id = Column(Integer, primary_key=True, autoincrement=True)
    brief_title = Column(
        "briefTitle",
        String(500),
        nullable=False,
    )
    nct_code = Column(String(11), nullable=False, unique=True)
    participant_count = Column(
        Integer,
        nullable=False,
        default=0,
        server_default="0",
    )
    sponsor_id = Column(
        Integer,
        ForeignKey("sponsors.id", ondelete="RESTRICT"),
        nullable=False,
```

```
            index=True,
        )
```

MySQL relationship evidence

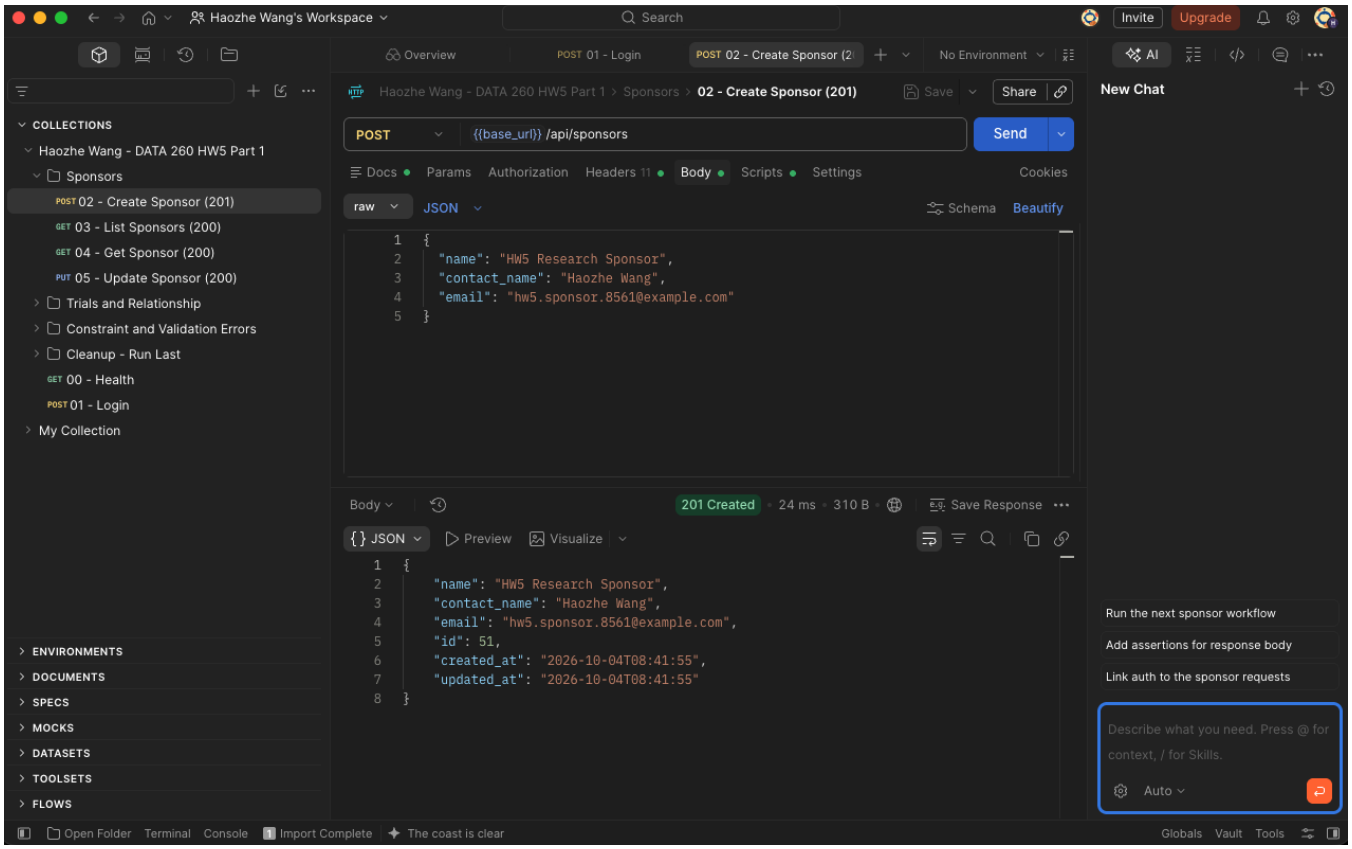


Relevant API code - Create Sponsor

```
def create_sponsor(
    sponsor_data: SponsorCreate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    sponsor = Sponsor(**sponsor_data.model_dump())
    db.add(sponsor)
    commit_or_conflict(db, "Sponsor email already exists")
    db.refresh(sponsor)
    return sponsor
```

```
@app.get(
    "/api/sponsors",
```

Postman evidence - Create Sponsor

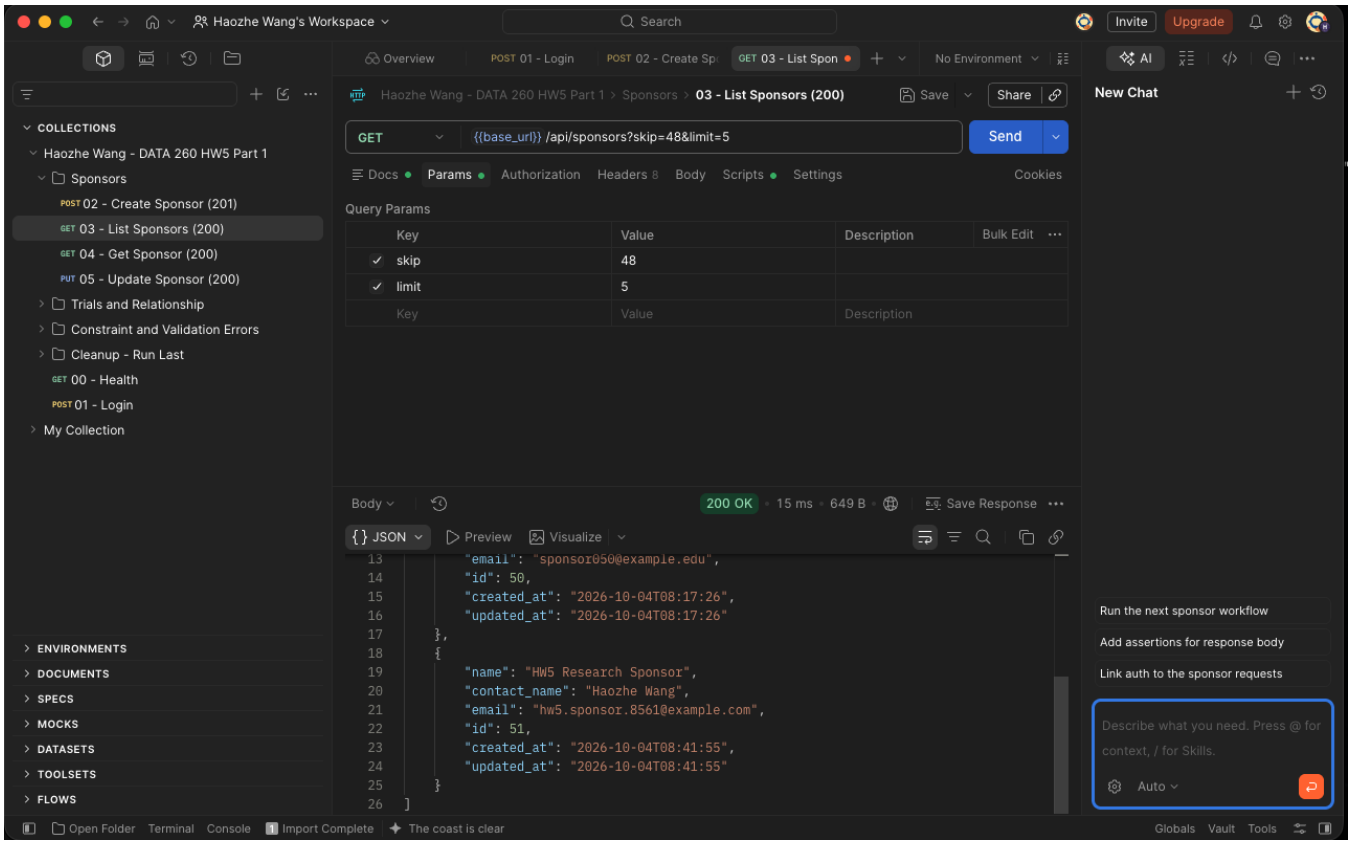


Relevant API code - List Sponsors

```
def get_sponsors(
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=50, ge=1, le=100),
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    return (
        db.query(Sponsor)
        .order_by(Sponsor.id.asc())
        .offset(skip)
        .limit(limit)
        .all()
```

)

Postman evidence - List Sponsors



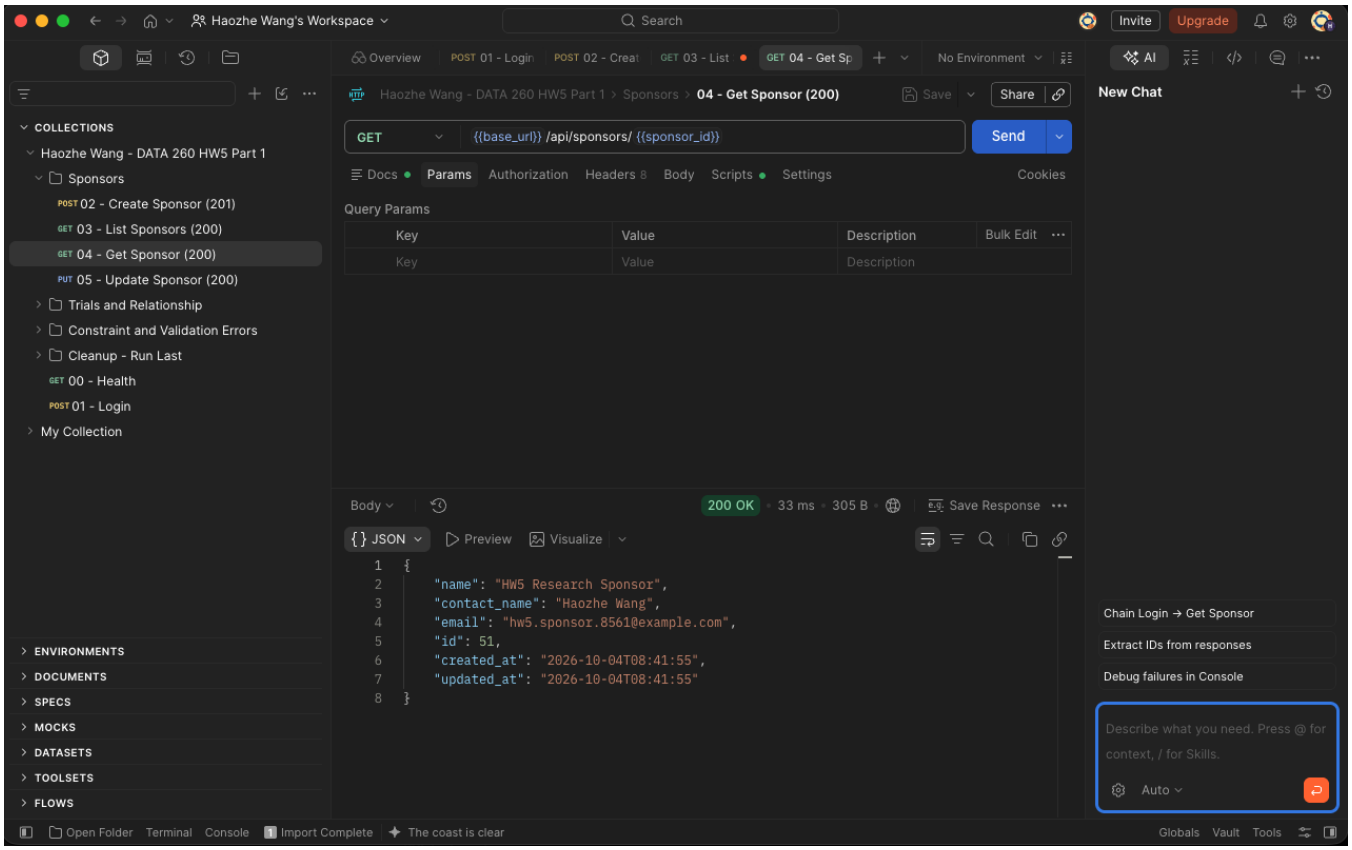
Relevant API code - Get Sponsor

```
def get_sponsor(
    sponsor_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    return find_sponsor_or_404(db, sponsor_id)


@app.put(
    "/api/sponsors/{sponsor_id}",
    response_model=SponsorOut,
    tags=["sponsors"],
```

```
)
def update_sponsor(
```

Postman evidence - Get Sponsor



Relevant API code - Update Sponsor

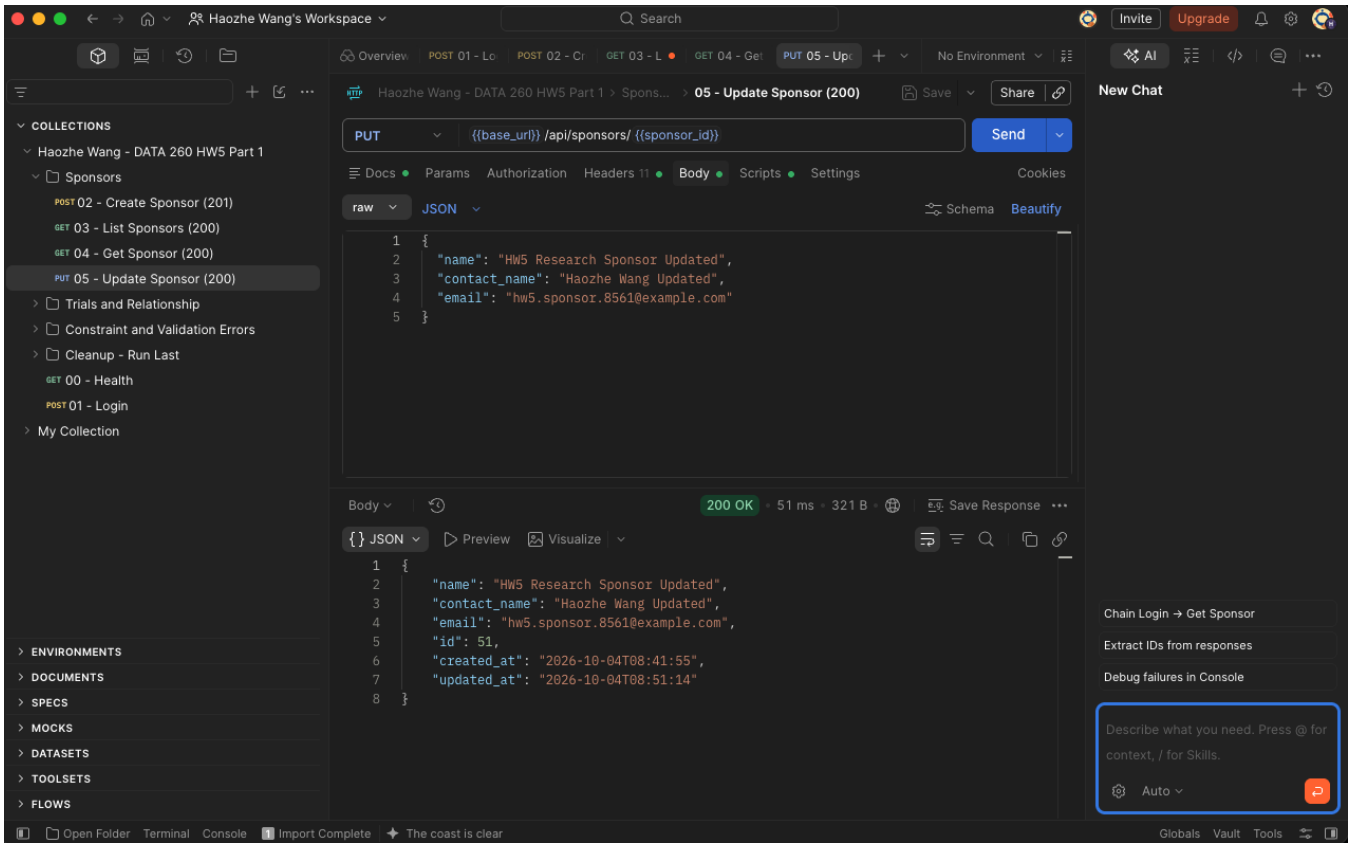
```
def update_sponsor(
    sponsor_id: int,
    sponsor_data: SponsorUpdate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    sponsor = find_sponsor_or_404(db, sponsor_id)

    for key, value in sponsor_data.model_dump().items():
        setattr(sponsor, key, value)

    commit_or_conflict(db, "Sponsor email already exists")
```

```
    db.refresh(sponsor)
    return sponsor
```

Postman evidence - Update Sponsor



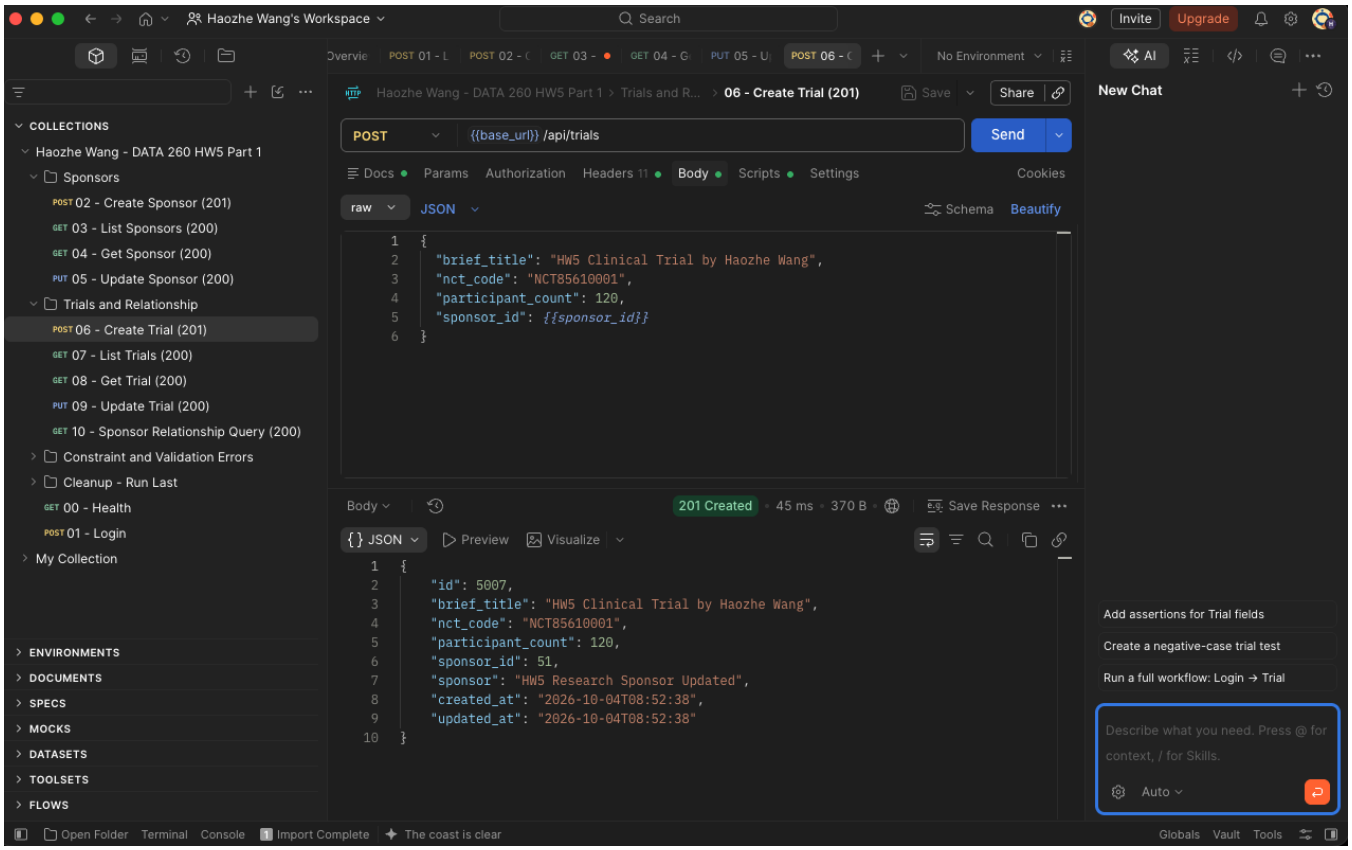
Relevant API code - Create Trial

```
def create_trial(
    trial_data: TrialCreate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    find_sponsor_or_404(db, trial_data.sponsor_id)
    trial = Trial(**trial_data.model_dump())

    db.add(trial)
    commit_or_conflict(db, "Trial NCT code already exists")
    db.refresh(trial)
```

```
    return trial
```

Postman evidence - Create Trial



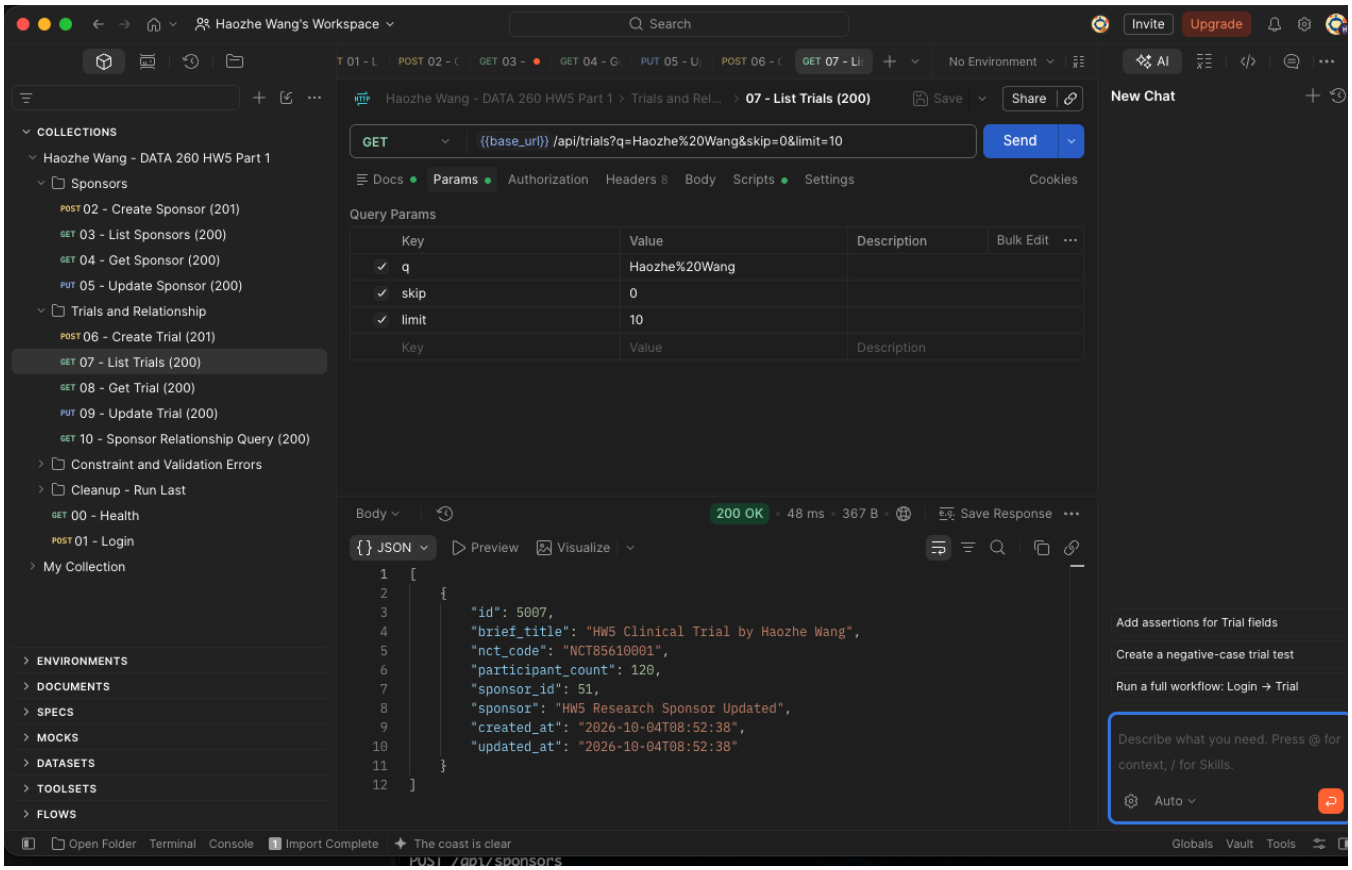
Relevant API code - List Trials

```
def get_trials(
    q: Optional[str] = Query(default=None),
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=50, ge=1, le=100),
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    query = db.query(Trial).options(joinedload(Trial.sponsor_entity))

    if q and q.strip():
        search_text = f"%{q.strip()}%"
        query = query.join(Trial.sponsor_entity).filter(
```

```
            (Trial.brief_title.ilike(search_text))
            | (Trial.nct_code.ilike(search_text))
```

Postman evidence - List Trials



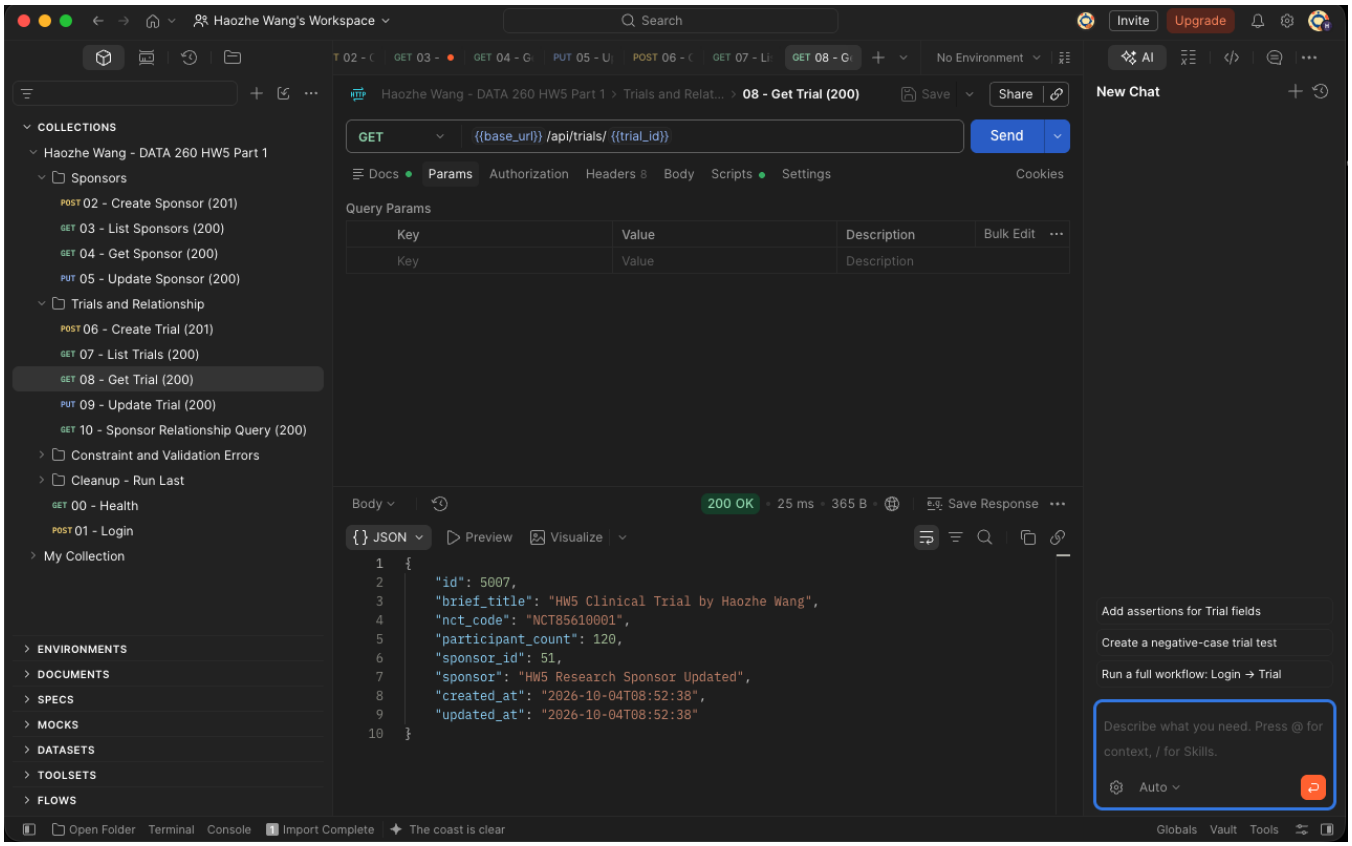
Relevant API code - Get Trial

```
def get_trial(
    trial_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    return find_trial_or_404(db, trial_id)


@app.post(
    "/api/trials",
    response_model=TrialOut,
    status_code=201,
```

```
        tags=["trials"],
    )
```

Postman evidence - Get Trial



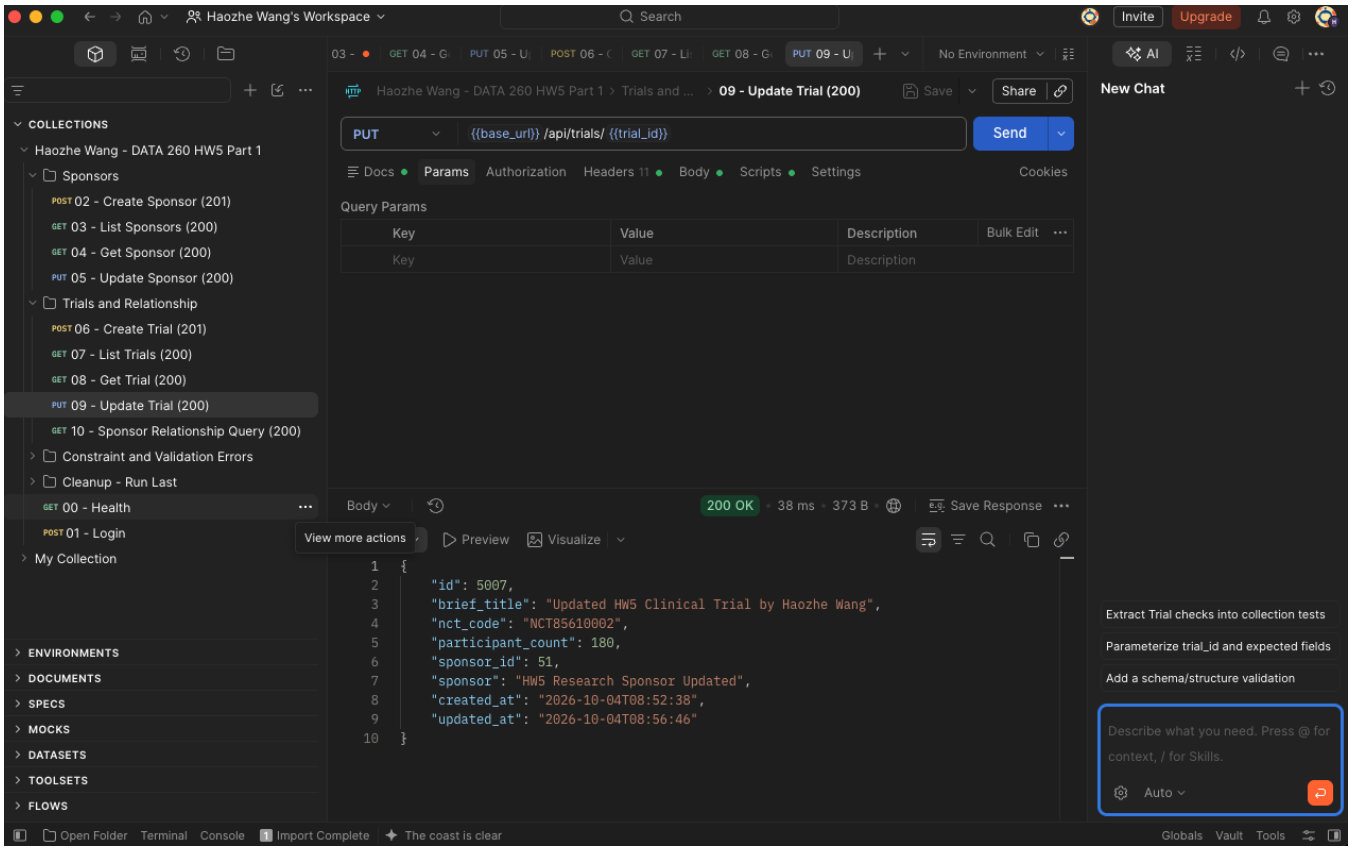
Relevant API code - Update Trial

```
def update_trial(
    trial_id: int,
    trial_data: TrialUpdate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    trial = find_trial_or_404(db, trial_id)
    find_sponsor_or_404(db, trial_data.sponsor_id)

    for key, value in trial_data.model_dump().items():
        setattr(trial, key, value)
```

```
    commit_or_conflict(db, "Trial NCT code already exists")
    db.refresh(trial)
```

Postman evidence - Update Trial

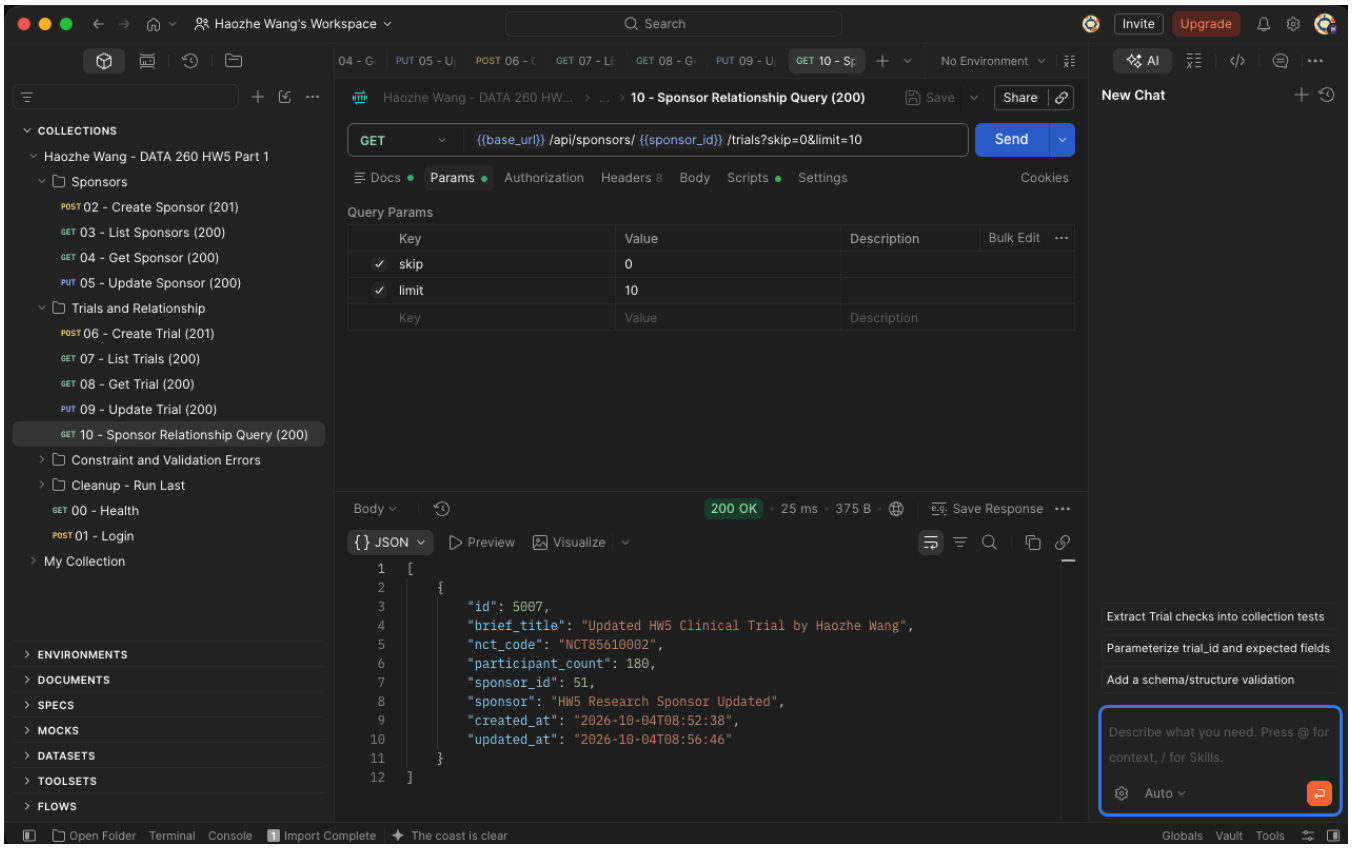


Relevant API code - Sponsor Trial Relationship

```
def get_sponsor_trials(
    sponsor_id: int,
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=50, ge=1, le=100),
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    find_sponsor_or_404(db, sponsor_id)
    return (
        db.query(Trial)
        .options(joinedload(Trial.sponsor_entity))
        .filter(Trial.sponsor_id == sponsor_id)
```

```
        .order_by(Trial.id.asc())
        .offset(skip)
```

Postman evidence - Sponsor Trial Relationship



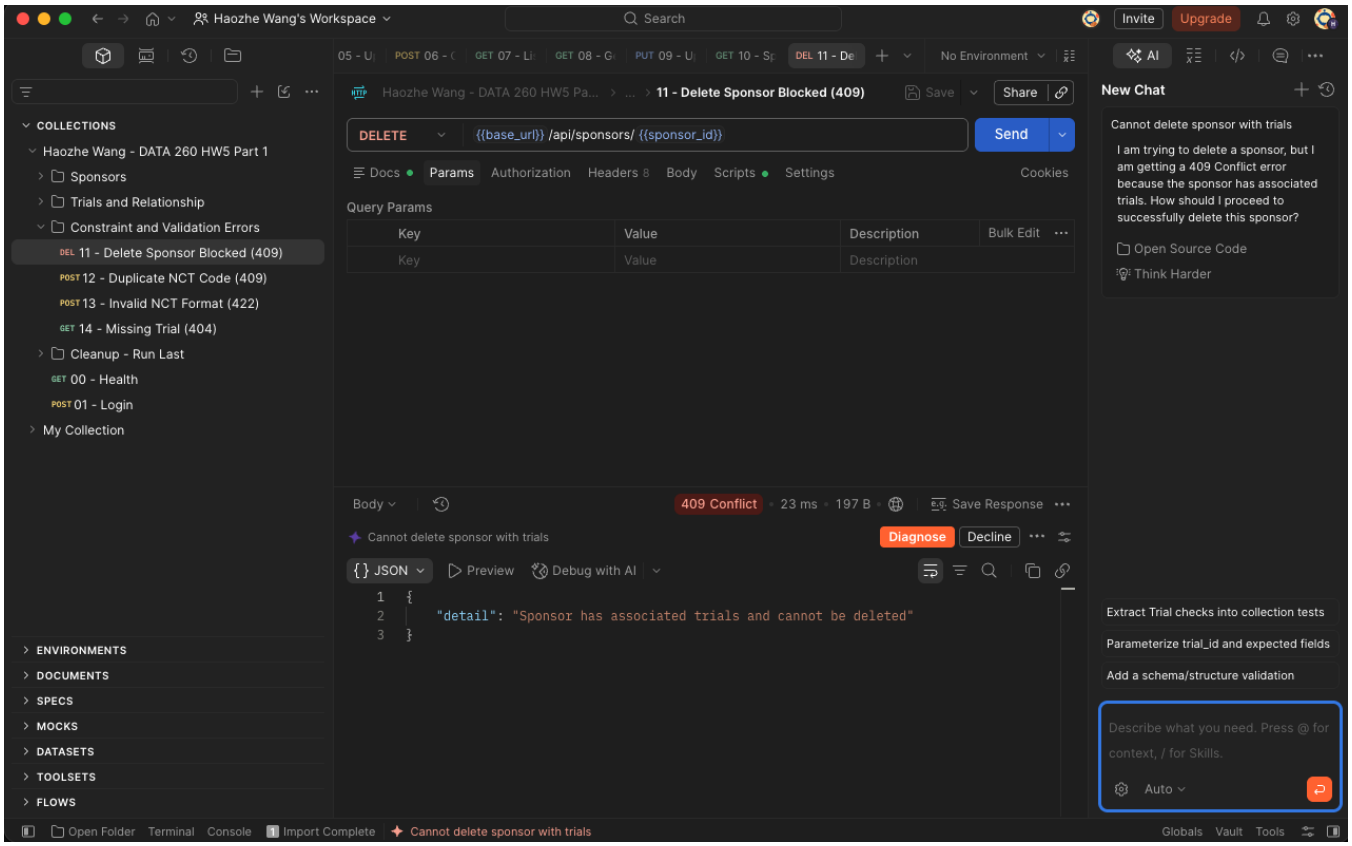
Relevant API code - Delete Sponsor Blocked 409

```
def delete_sponsor(
    sponsor_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    sponsor = find_sponsor_or_404(db, sponsor_id)

    if db.query(Trial.id).filter(Trial.sponsor_id == sponsor_id).first():
        raise HTTPException(
            status_code=409,
            detail="Sponsor has associated trials and cannot be deleted",
        )
```

```
    db.delete(sponsor)
```

Postman evidence - Delete Sponsor Blocked 409



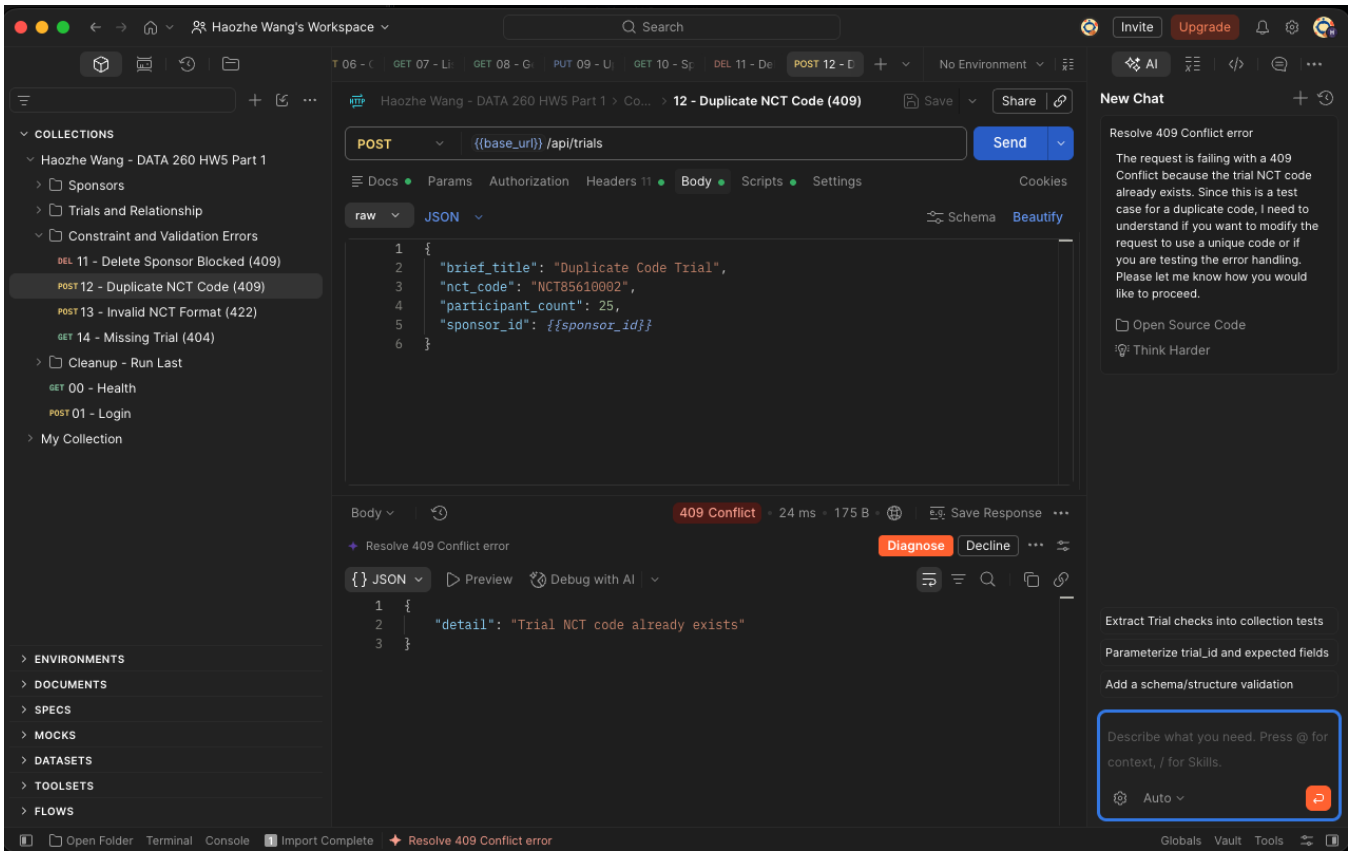
Relevant API code - Duplicate NCT Code 409

```
def create_trial(
    trial_data: TrialCreate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    find_sponsor_or_404(db, trial_data.sponsor_id)
    trial = Trial(**trial_data.model_dump())

    db.add(trial)
    commit_or_conflict(db, "Trial NCT code already exists")
    db.refresh(trial)
```

```
    return trial
```

Postman evidence - Duplicate NCT Code 409

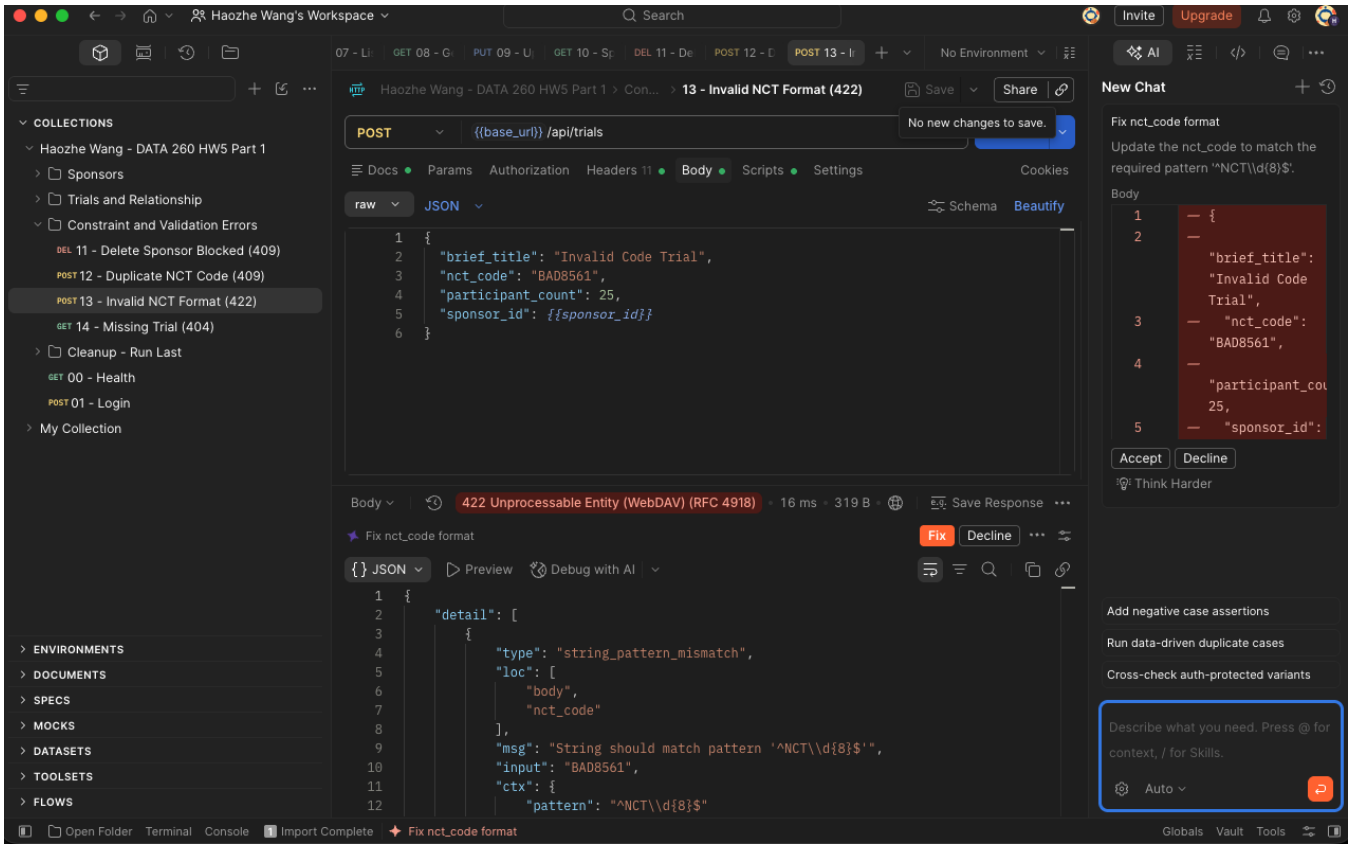


Relevant API code - Invalid NCT Code 422

```
class TrialCreate(BaseModel):
    brief_title: str = Field(min_length=1, max_length=500)
    nct_code: str = Field(pattern=r"^NCT\d{8}$")
    participant_count: int = Field(default=0, ge=0)
    sponsor_id: int = Field(gt=0)


class TrialUpdate(BaseModel):
    brief_title: str = Field(min_length=1, max_length=500)
    nct_code: str = Field(pattern=r"^NCT\d{8}$")
    participant_count: int = Field(ge=0)
    sponsor_id: int = Field(gt=0)
```

Postman evidence - Invalid NCT Code 422



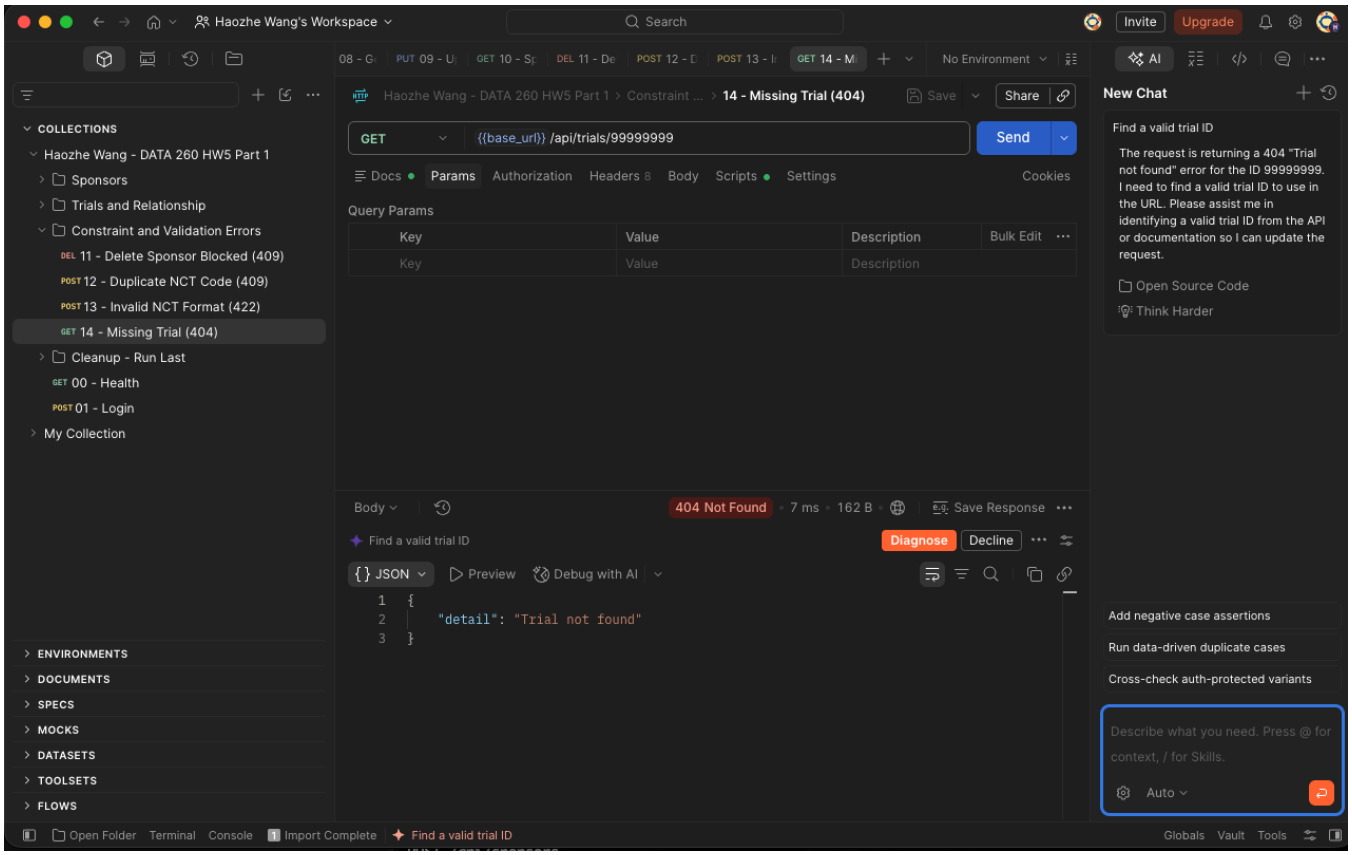
Relevant API code - Missing Trial 404

```
def find_trial_or_404(db: Session, trial_id: int):
    trial = (
        db.query(Trial)
        .options(joinedload(Trial.sponsor_entity))
        .filter(Trial.id == trial_id)
        .first()
    )

    if not trial:
        raise HTTPException(status_code=404, detail="Trial not found")

    return trial
```

Postman evidence - Missing Trial 404



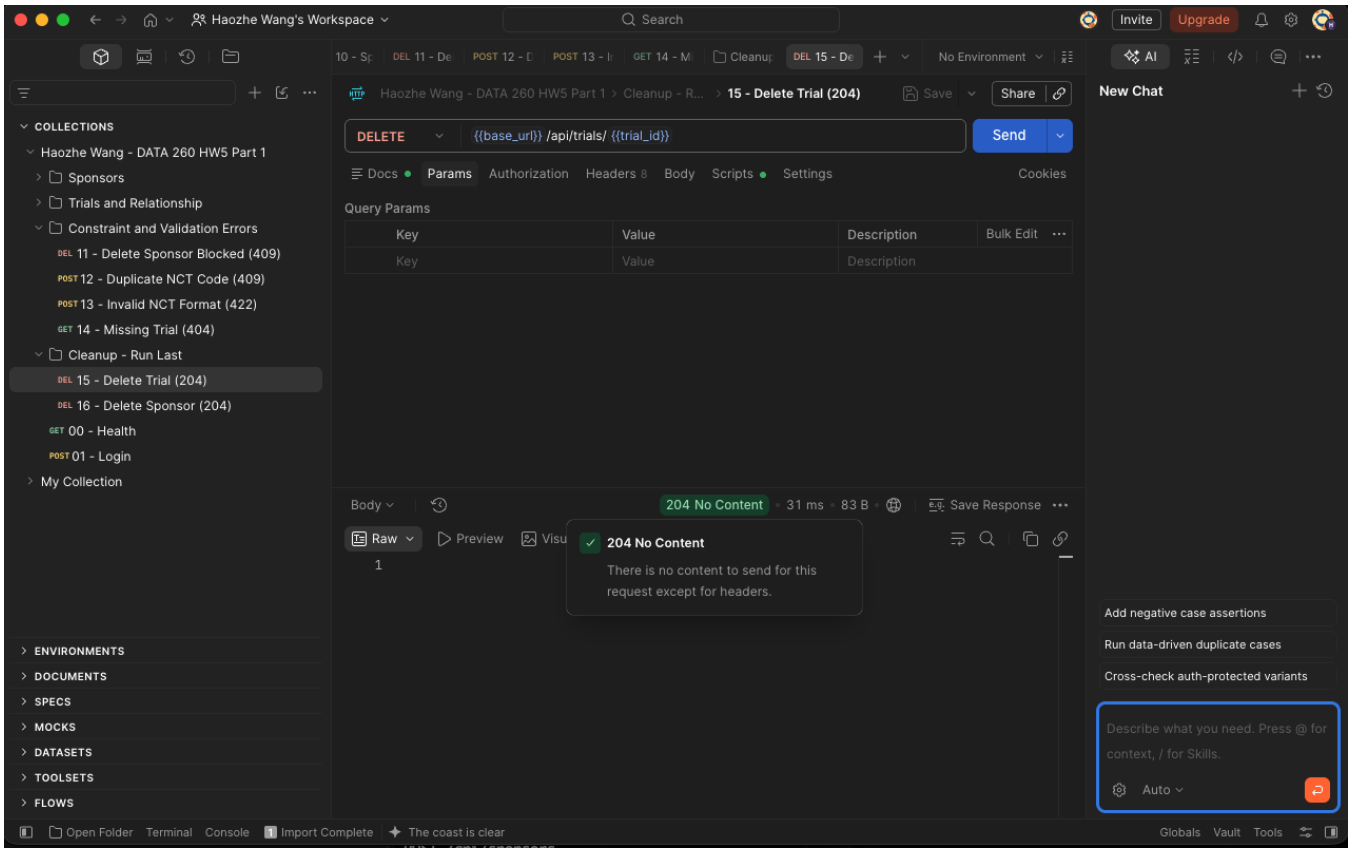
Relevant API code - Delete Trial 204

```
def delete_trial(
    trial_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    trial = find_trial_or_404(db, trial_id)

    db.delete(trial)
    commit_or_conflict(db, "Trial could not be deleted")
    return Response(status_code=204)
```

```
@app.get("/health")
def health():
```

Postman evidence - Delete Trial 204



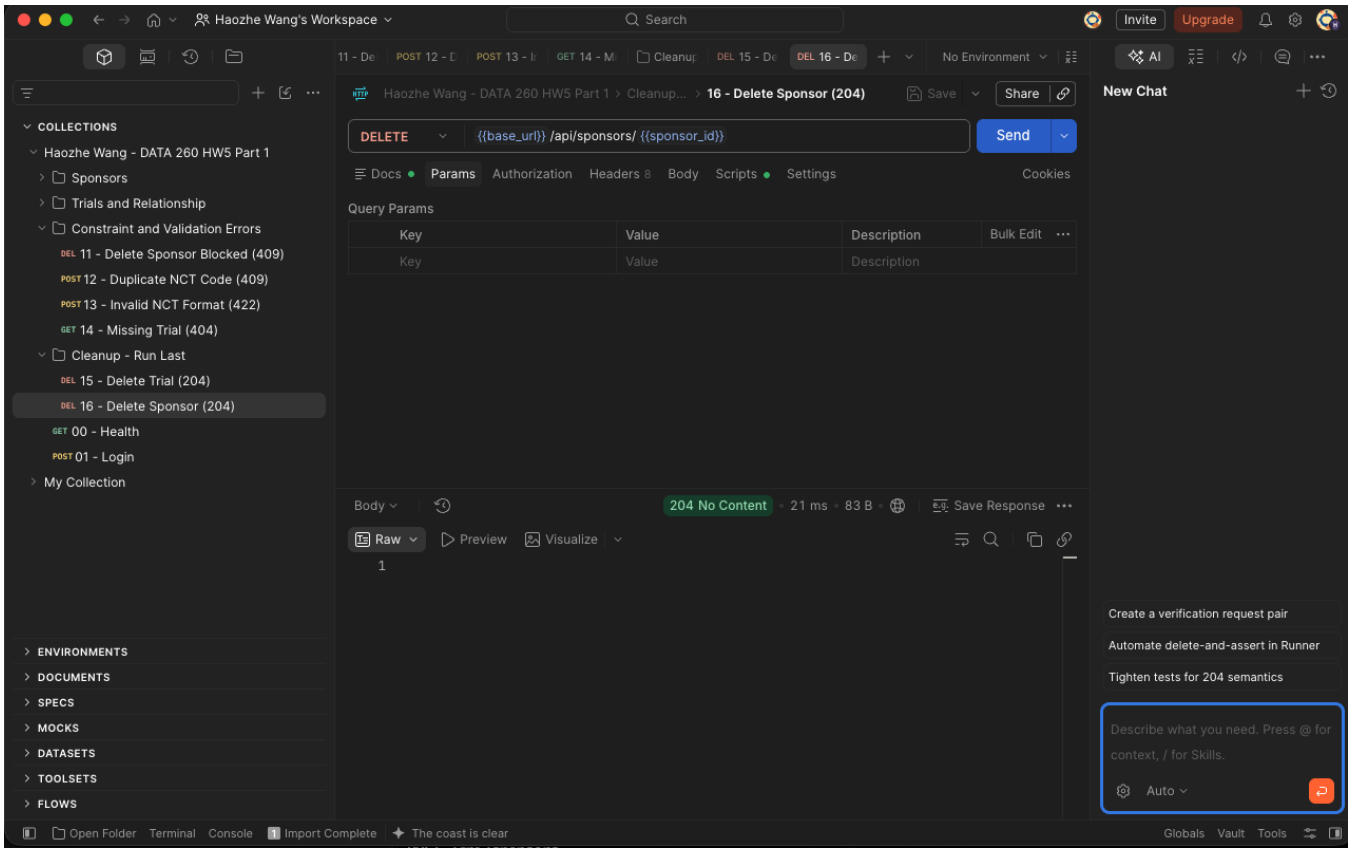
Relevant API code - Delete Sponsor 204

```
def delete_sponsor(
    sponsor_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    sponsor = find_sponsor_or_404(db, sponsor_id)

    if db.query(Trial.id).filter(Trial.sponsor_id == sponsor_id).first():
        raise HTTPException(
            status_code=409,
            detail="Sponsor has associated trials and cannot be deleted",
        )
```

```
db.delete(sponsor)
```

Postman evidence - Delete Sponsor 204



Part 1 Redux Toolkit Client

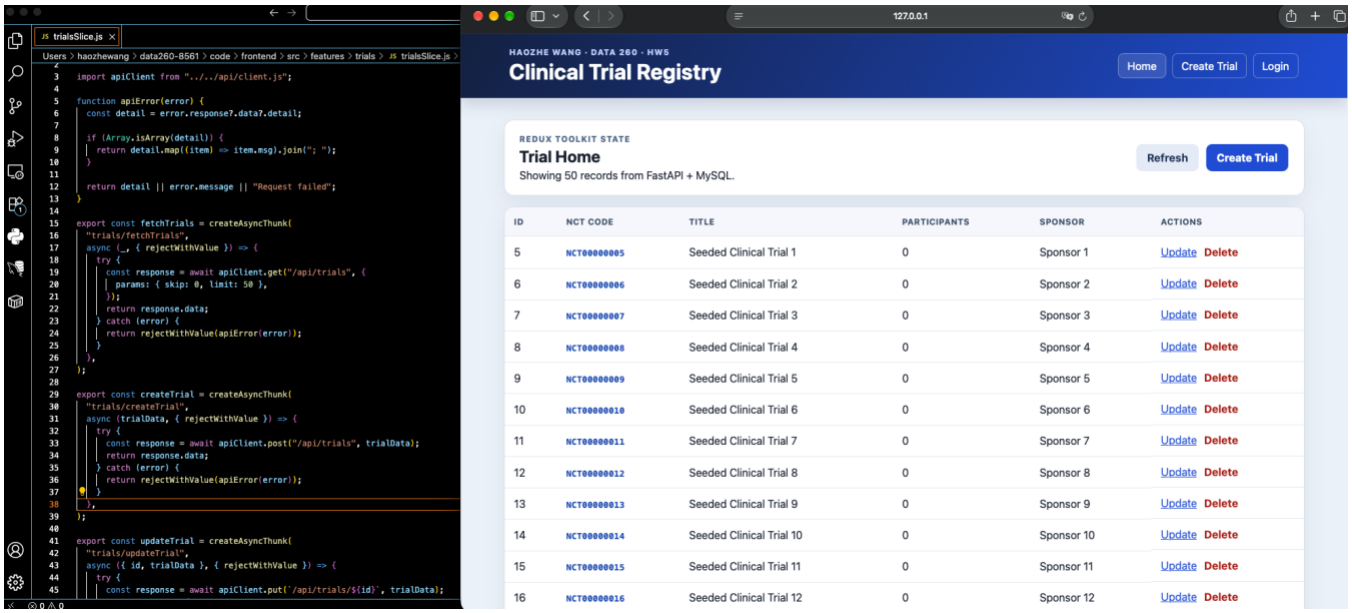
Redux Toolkit owns the Trial state. Axios backed async thunks fetch, create, update, and delete records, and fulfilled actions update the Home screen.

Redux thunk - Home and Fetch

```
export const fetchTrials = createAsyncThunk(
  "trials/fetchTrials",
  async (_, { rejectWithValue }) => {
    try {
      const response = await apiClient.get("/api/trials", {
        params: { skip: 0, limit: 50 },
      });
      return response.data;
    } catch (error) {
      return rejectWithValue(apiError(error));
    }
  },
);

export const createTrial = createAsyncThunk(
  "trials/createTrial",
```

Redux UI evidence - Home and Fetch



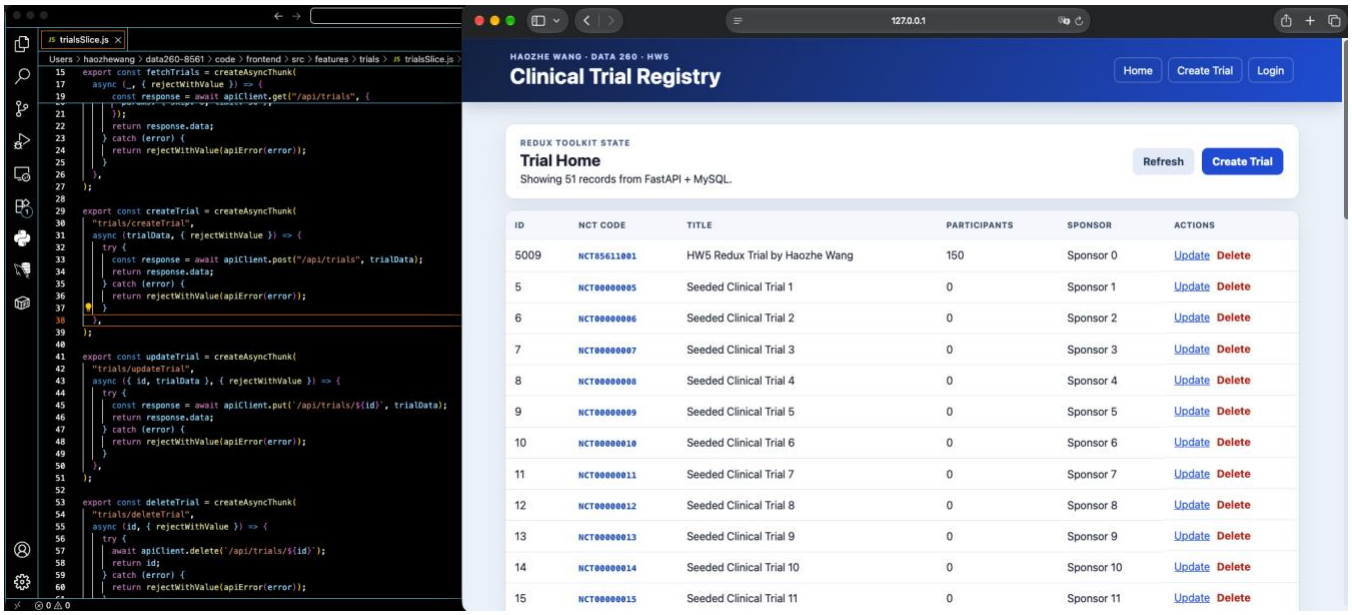
Redux thunk - Create

```
export const createTrial = createAsyncThunk(
  "trials/createTrial",
  async (trialData, { rejectWithValue }) => {
    try {
      const response = await apiClient.post("/api/trials", trialData);
      return response.data;
    } catch (error) {
      return rejectWithValue(apiError(error));
    }
  },
);

export const updateTrial = createAsyncThunk(
  "trials/updateTrial",
```

```
  async ({ id, trialData }, { rejectWithValue }) => {
    try {
```

Redux UI evidence - Create

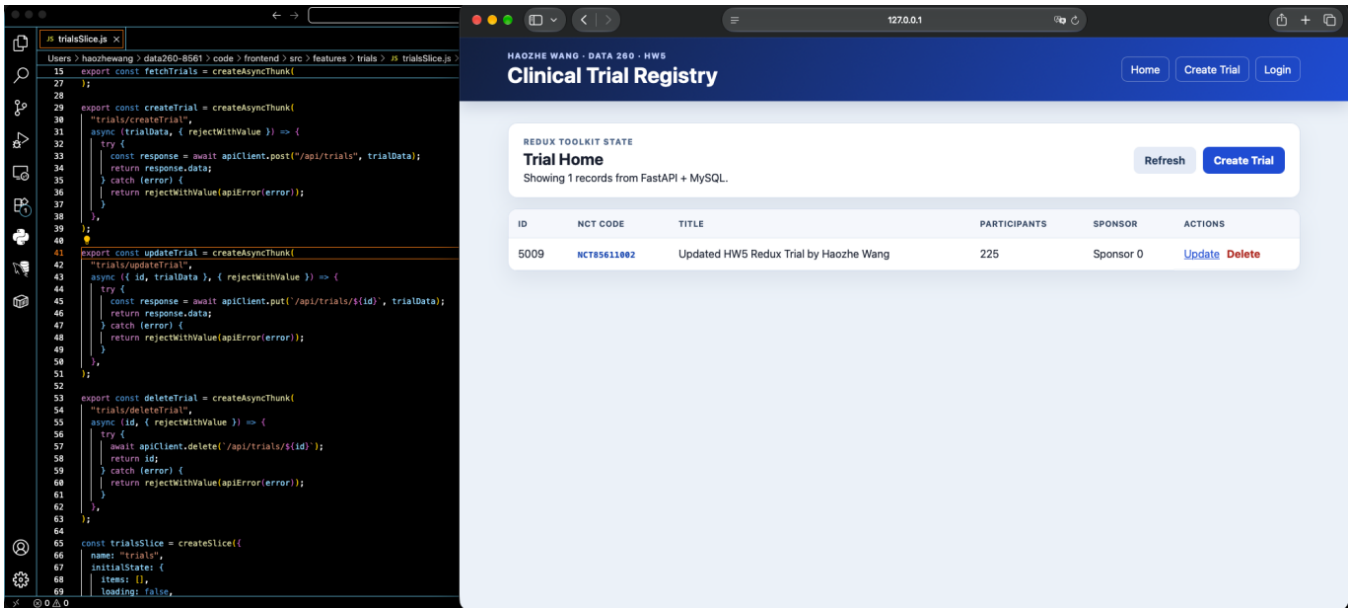


Redux thunk - Update

```
export const updateTrial = createAsyncThunk(
  "trials/updateTrial",
  async ({ id, trialData }, { rejectWithValue }) => {
    try {
      const response = await apiClient.put(`/api/trials/${id}`, trialData);
      return response.data;
    } catch (error) {
      return rejectWithValue(apiError(error));
    }
  },
);

export const deleteTrial = createAsyncThunk(
  "trials/deleteTrial",
  async (id, { rejectWithValue }) => {
    try {
```

Redux UI evidence - Update

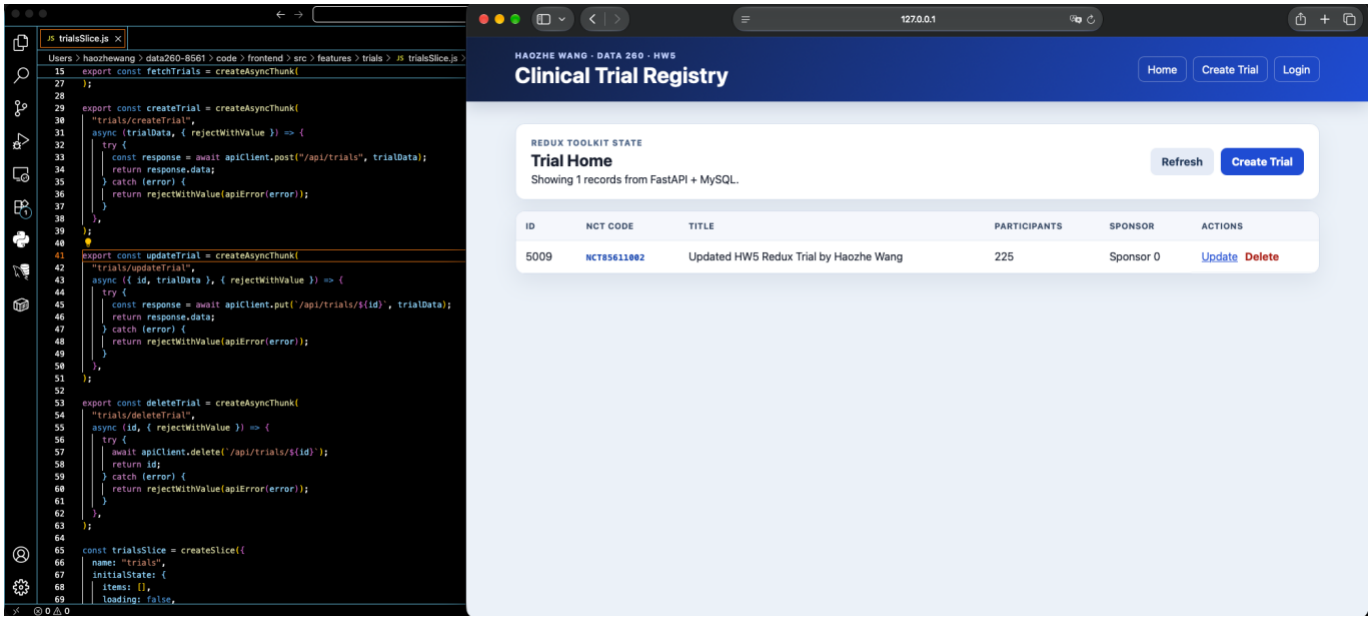


Redux thunk - Delete

```
export const deleteTrial = createAsyncThunk(
  "trials/deleteTrial",
  async (id, { rejectWithValue }) => {
    try {
      await apiClient.delete(`/api/trials/${id}`);
      return id;
    } catch (error) {
      return rejectWithValue(apiError(error));
    }
  },
);

const trialsSlice = createSlice({
  name: "trials",
  initialState: {
    items: [],
```

Redux UI evidence - Delete



Part 2 MCP Tool Servers

The Meals STDIO server exposes four TheMealDB tools. The domain STDIO server exposes exactly three clinical trial tools with the shared ok data error envelope.

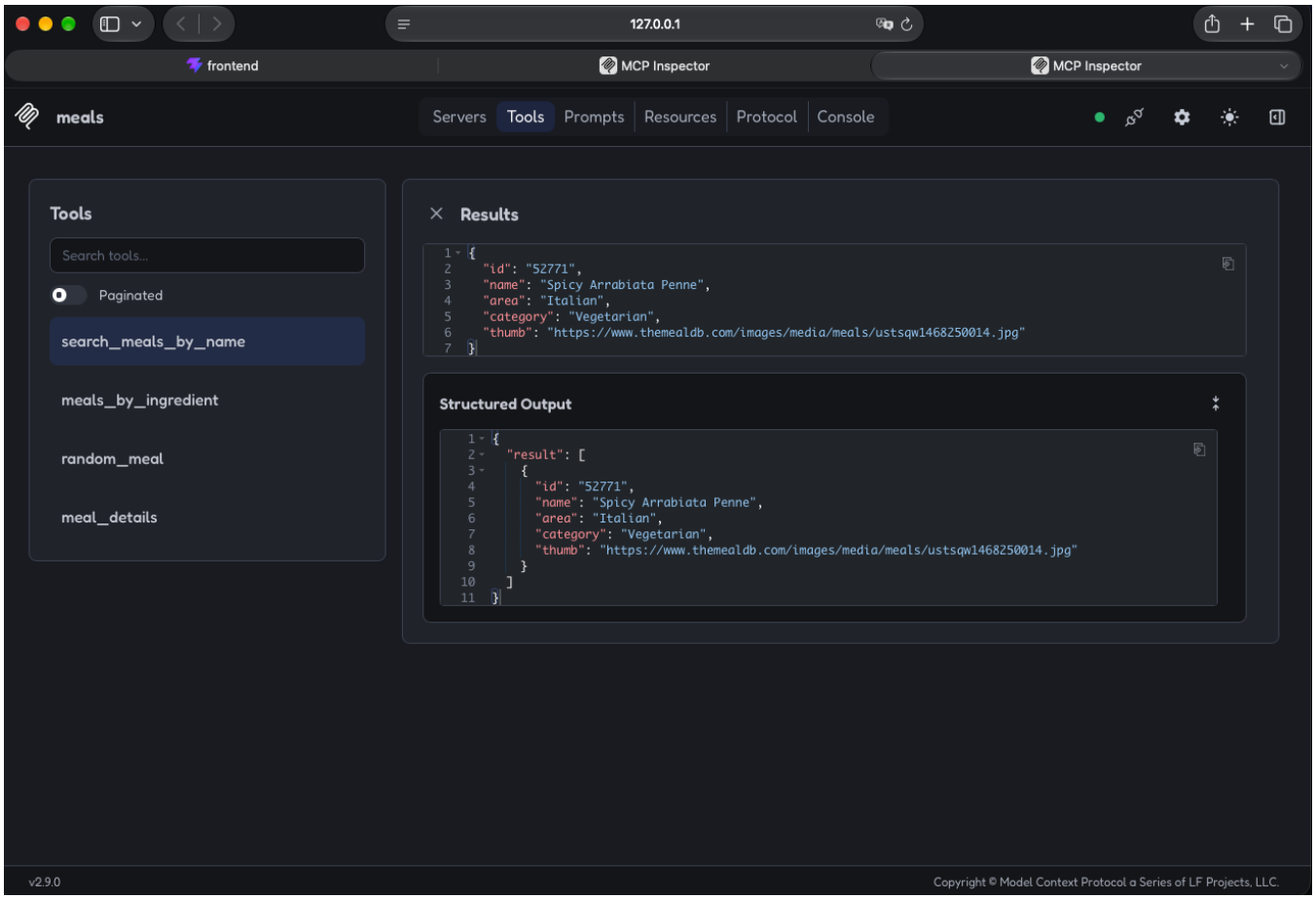
Meals MCP tool - search_meals_by_name

```
@mcp.tool()
async def search_meals_by_name(
    query: str, limit: int = 5
) -> list[dict[str, Any]] | dict[str, Any]:
    """Search meal names and return up to 25 compact recipe summaries."""
    clean_query = _required_text(query, "query")
    clean_limit = _bounded_limit(limit)
    rows = _meal_rows(await _fetch("search.php", {"s": clean_query}))
    if rows is None:
        return {"results": [], "message": "no matches"}
    return [_summary(row) for row in rows[:clean_limit]]


@mcp.tool()
```

```
async def meals_by_ingredient(
    ingredient: str, limit: int = 12
```

MCP Inspector - search_meals_by_name



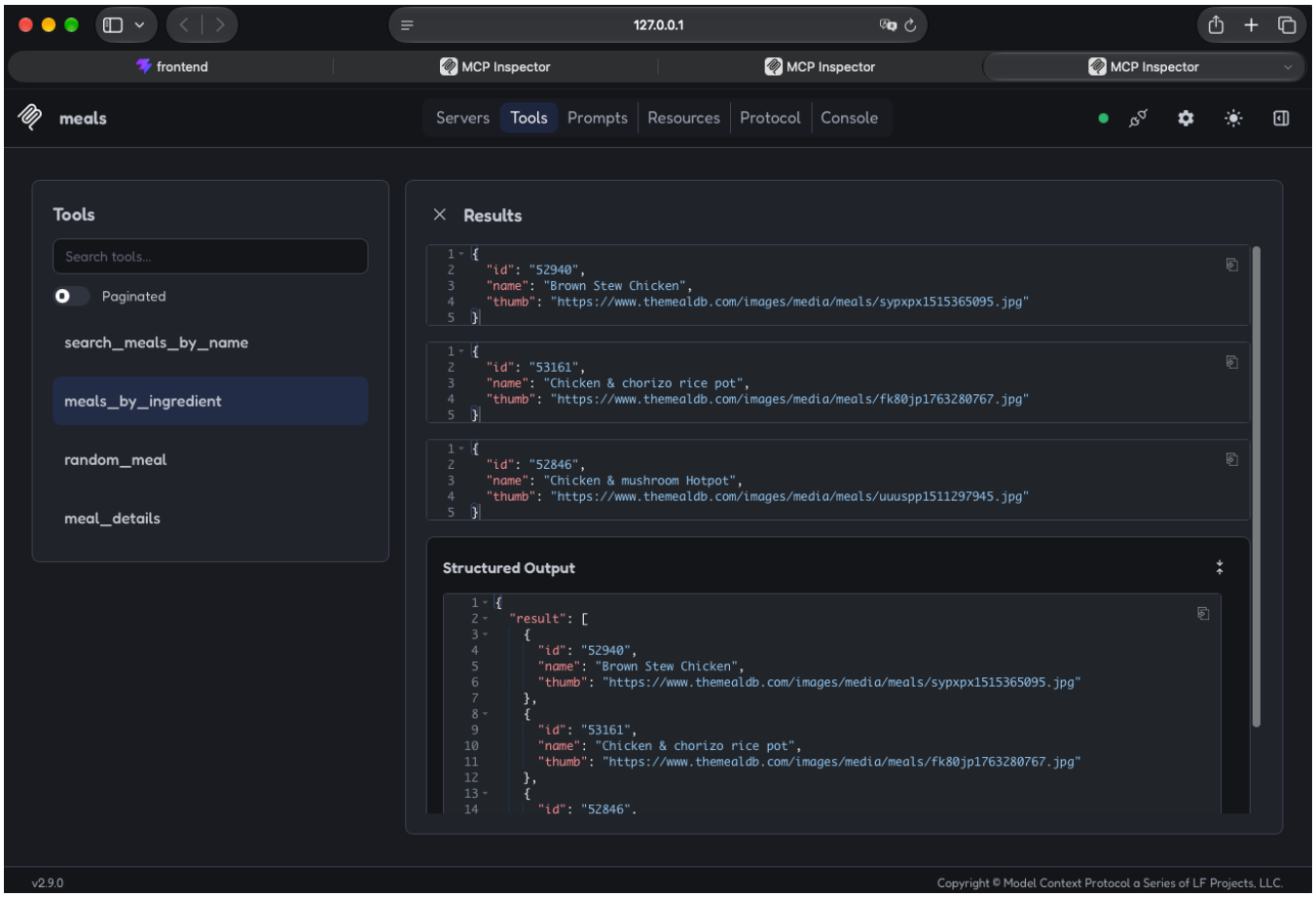
Meals MCP tool - meals_by_ingredient

```
@mcp.tool()
async def meals_by_ingredient(
    ingredient: str, limit: int = 12
) -> list[dict[str, Any]] | dict[str, Any]:
    """Find meals by one main ingredient and return compact recipe cards."""
    clean_ingredient = _required_text(ingredient, "ingredient")
    clean_limit = _bounded_limit(limit)
    rows = _meal_rows(await _fetch("filter.php", {"i": clean_ingredient}))
    if rows is None:
        return {"results": [], "message": "no matches"}
    return [_ingredient_card(row) for row in rows[:clean_limit]]


@mcp.tool()
```

```
async def random_meal() -> dict[str, Any]:
    """Return one random meal with full recipe details."""
```

MCP Inspector - meals_by_ingredient



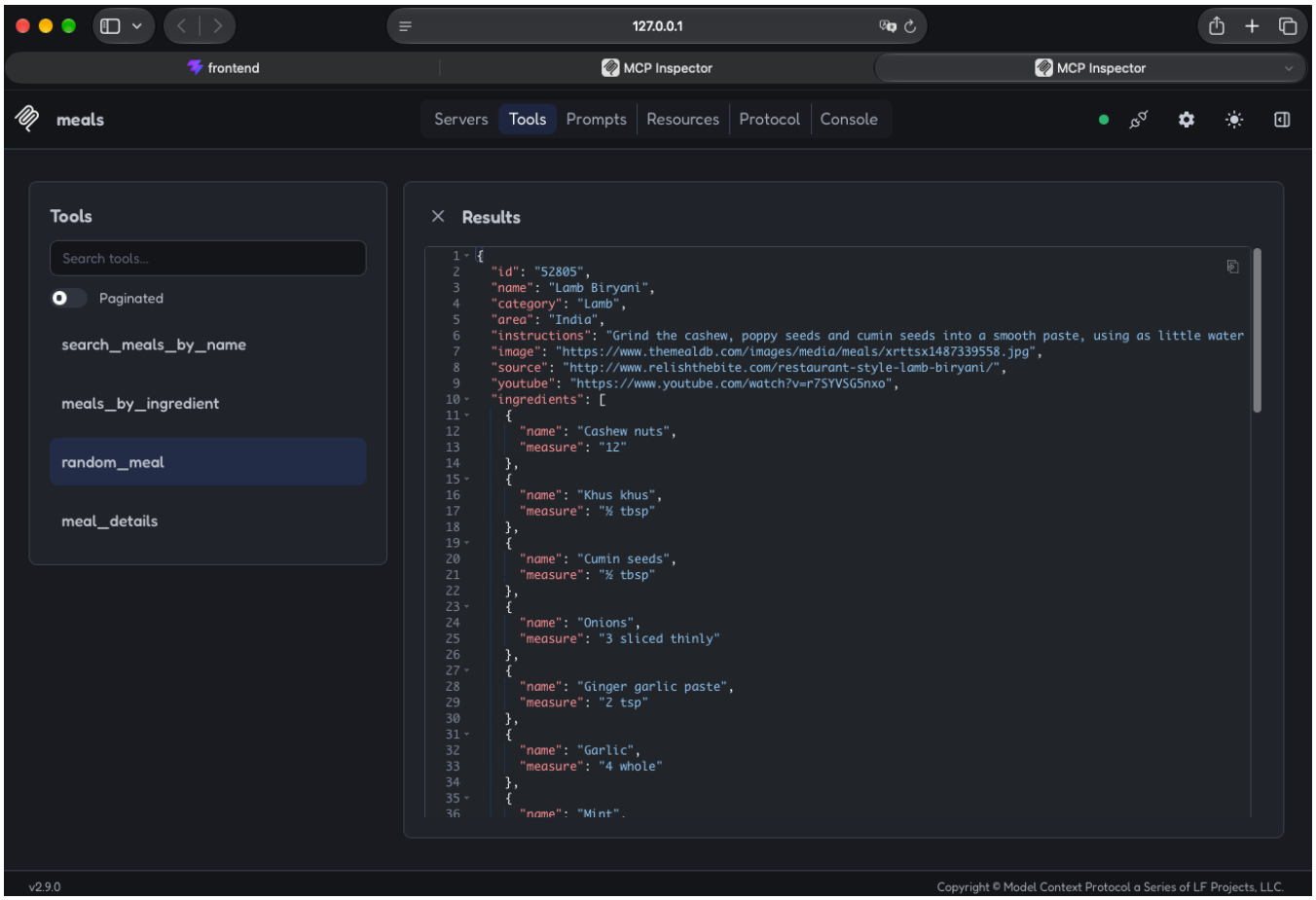
Meals MCP tool - random_meal

```
@mcp.tool()
async def random_meal() -> dict[str, Any]:
    """Return one random meal with full recipe details."""
    rows = _meal_rows(await _fetch("random.php"))
    if not rows:
        return {"result": None, "message": "no matches"}
    return _details(rows[0])


@mcp.tool()
async def meal_details(id: str | int) -> dict[str, Any]:
    """Look up one meal id and return its full recipe details."""
    clean_id = _meal_id(id)
    rows = _meal_rows(await _fetch("lookup.php", {"i": clean_id}))
```

```
    if not rows:
        return {"result": None, "message": "no matches"}
```

MCP Inspector - random_meal

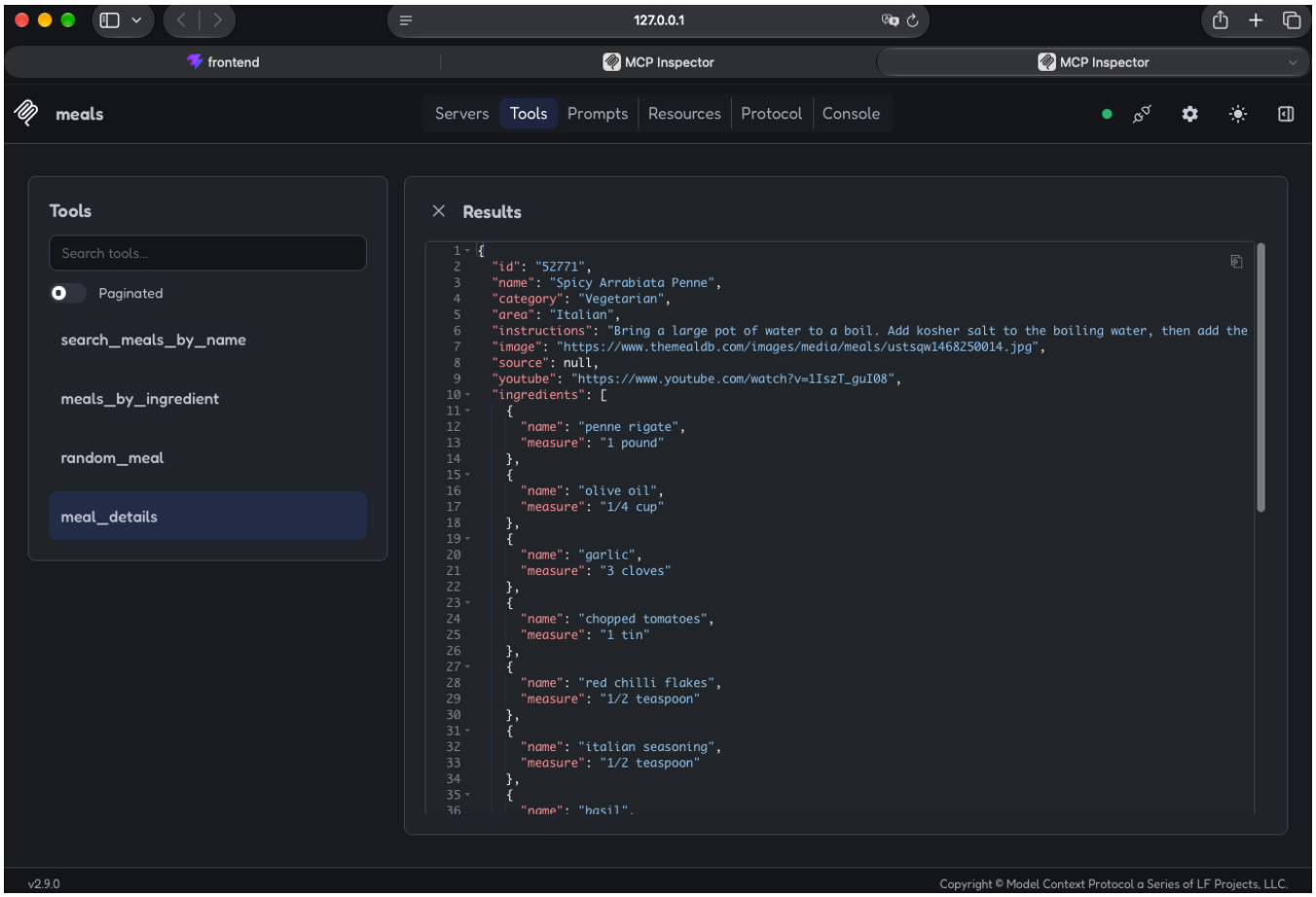


Meals MCP tool - meal_details

```
@mcp.tool()
async def meal_details(id: str | int) -> dict[str, Any]:
    """Look up one meal id and return its full recipe details."""
    clean_id = _meal_id(id)
    rows = _meal_rows(await _fetch("lookup.php", {"i": clean_id}))
    if not rows:
        return {"result": None, "message": "no matches"}
    return _details(rows[0])
```

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

MCP Inspector - meal_details



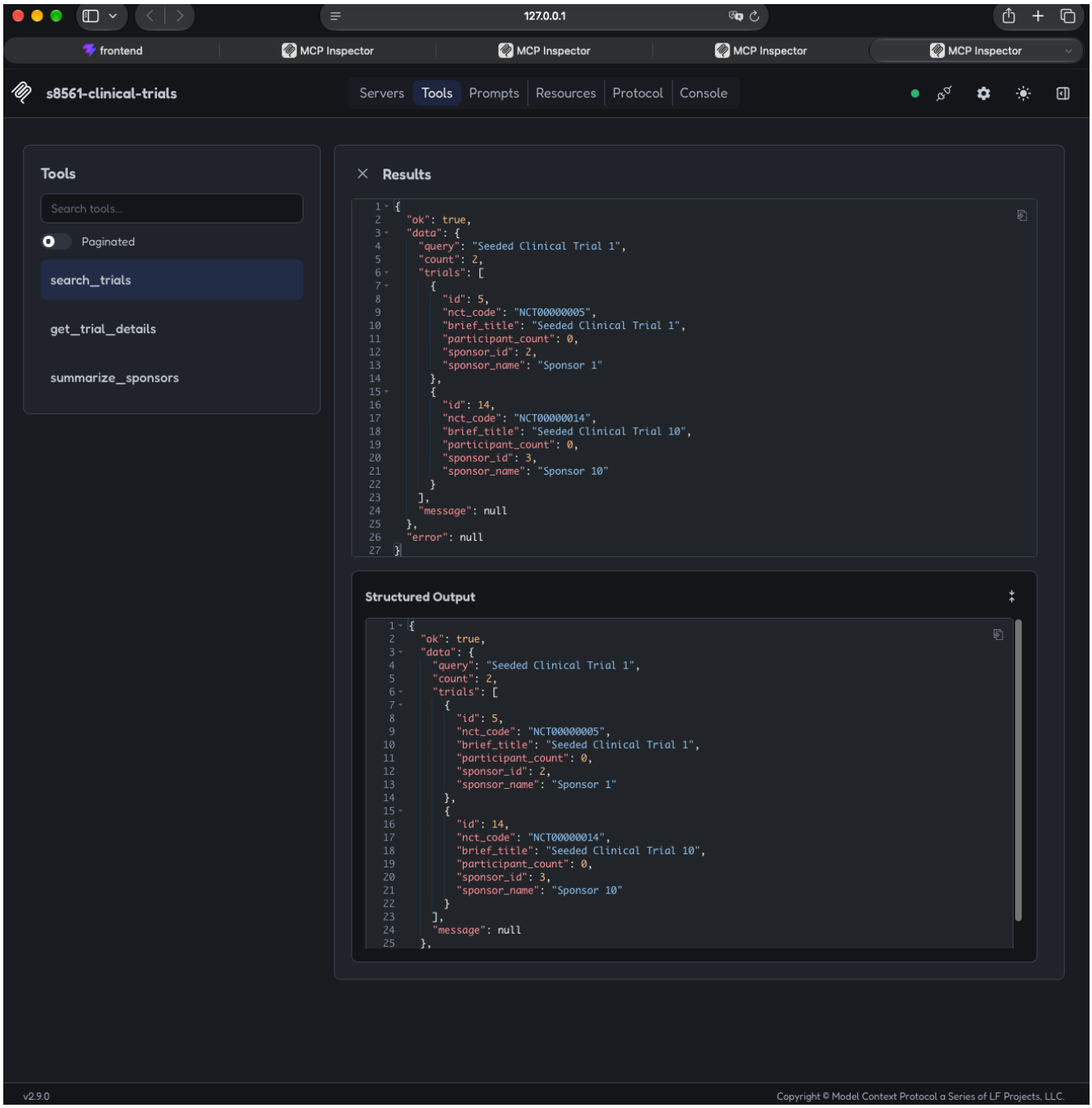
Domain MCP tool - search_trials success

```
def search_trials_data(
    query: str,
    limit: int = 10,
    *,
    session_factory: SessionFactory = SessionLocal,
) -> dict[str, Any]:
    """Implementation for searching trials, injectable for offline tests."""
    if not isinstance(query, str) or not query.strip():
        return failure("query must be a non-empty string")
    if isinstance(limit, bool) or not isinstance(limit, int) or not 1 <= limit <= 25:
        return failure("limit must be an integer between 1 and 25")

    clean_query = query.strip()
    match = f"%{clean_query}%"
    def operation() -> list[dict[str, Any]]:
```

```
        with session_factory() as session:
            statement = (
```

MCP Inspector - search_trials success



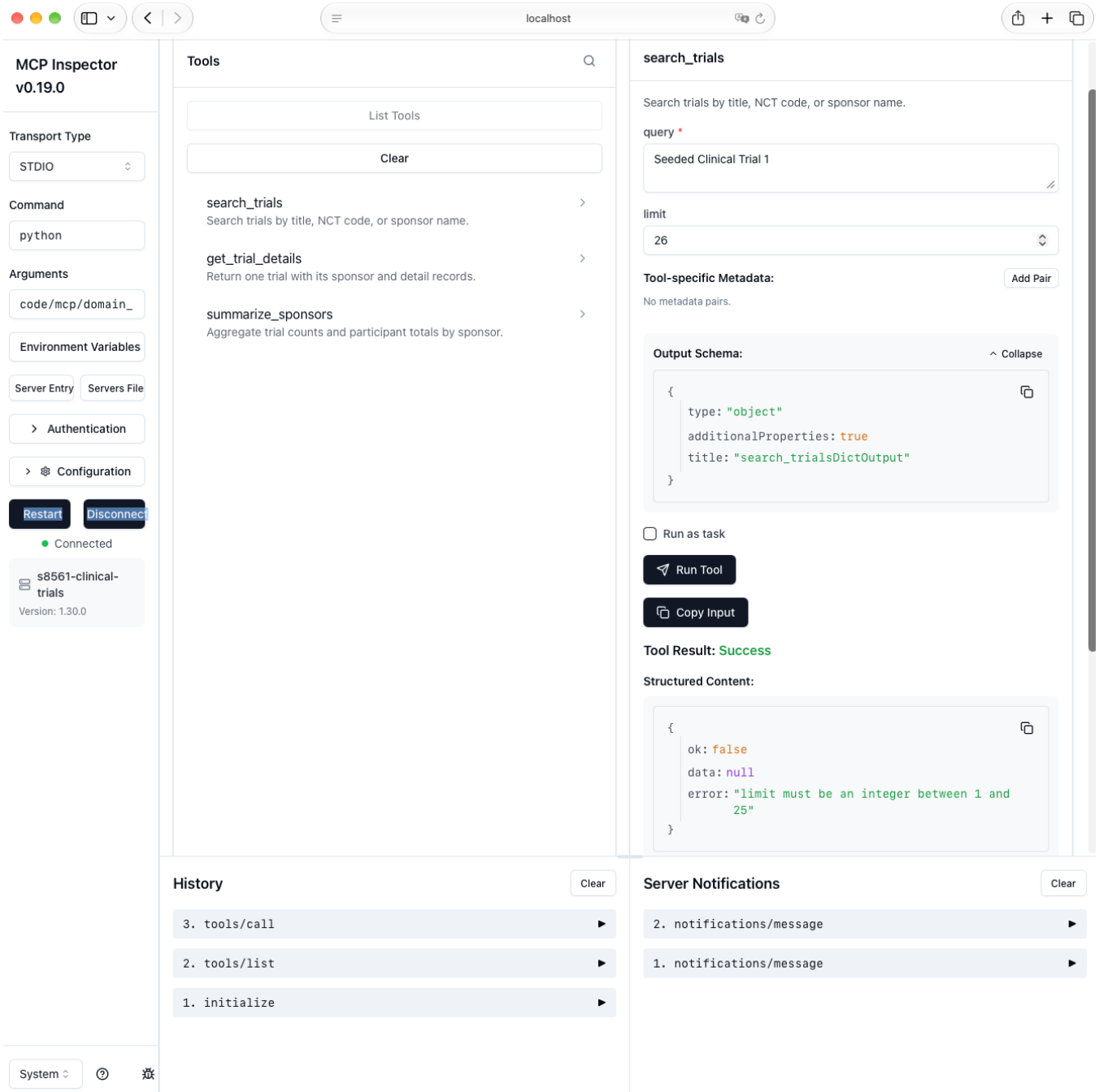
Domain MCP tool - search_trials invalid

```
def search_trials_data(
    query: str,
    limit: int = 10,
    *,
    session_factory: SessionFactory = SessionLocal,
) -> dict[str, Any]:
    """Implementation for searching trials, injectable for offline tests."""
    if not isinstance(query, str) or not query.strip():
        return failure("query must be a non-empty string")
    if isinstance(limit, bool) or not isinstance(limit, int) or not 1 <= limit <= 25:
        return failure("limit must be an integer between 1 and 25")

    clean_query = query.strip()
    match = f"%{clean_query}%"
```

```
    def operation() -> list[dict[str, Any]]:
        with session_factory() as session:
            statement = (
```

MCP Inspector - search_trials invalid



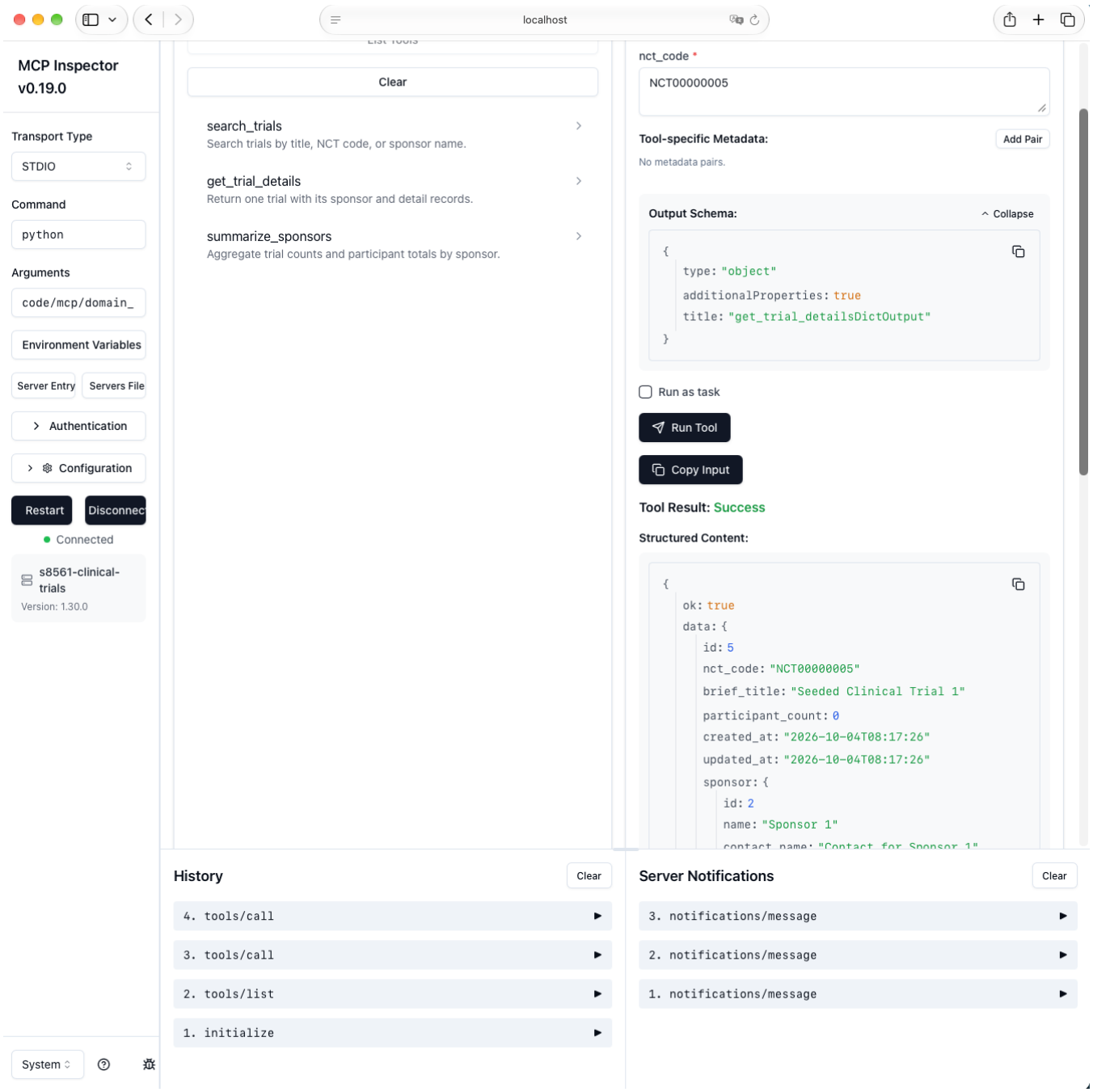
Domain MCP tool - get_trial_details success

```
def get_trial_details_data(
    nct_code: str,
    *,
    session_factory: SessionFactory = SessionLocal,
) -> dict[str, Any]:
    """Implementation for one complete trial lookup."""
    if not isinstance(nct_code, str):
        return failure("nct_code must be a string in NCT######## format")
    clean_code = nct_code.strip().upper()
    if not NCT_PATTERN.fullmatch(clean_code):
        return failure("nct_code must match NCT########")

    def operation() -> dict[str, Any] | None:
```

```
        with session_factory() as session:
            statement = (
                select(Trial)
                .options(
```

MCP Inspector - get_trial_details success

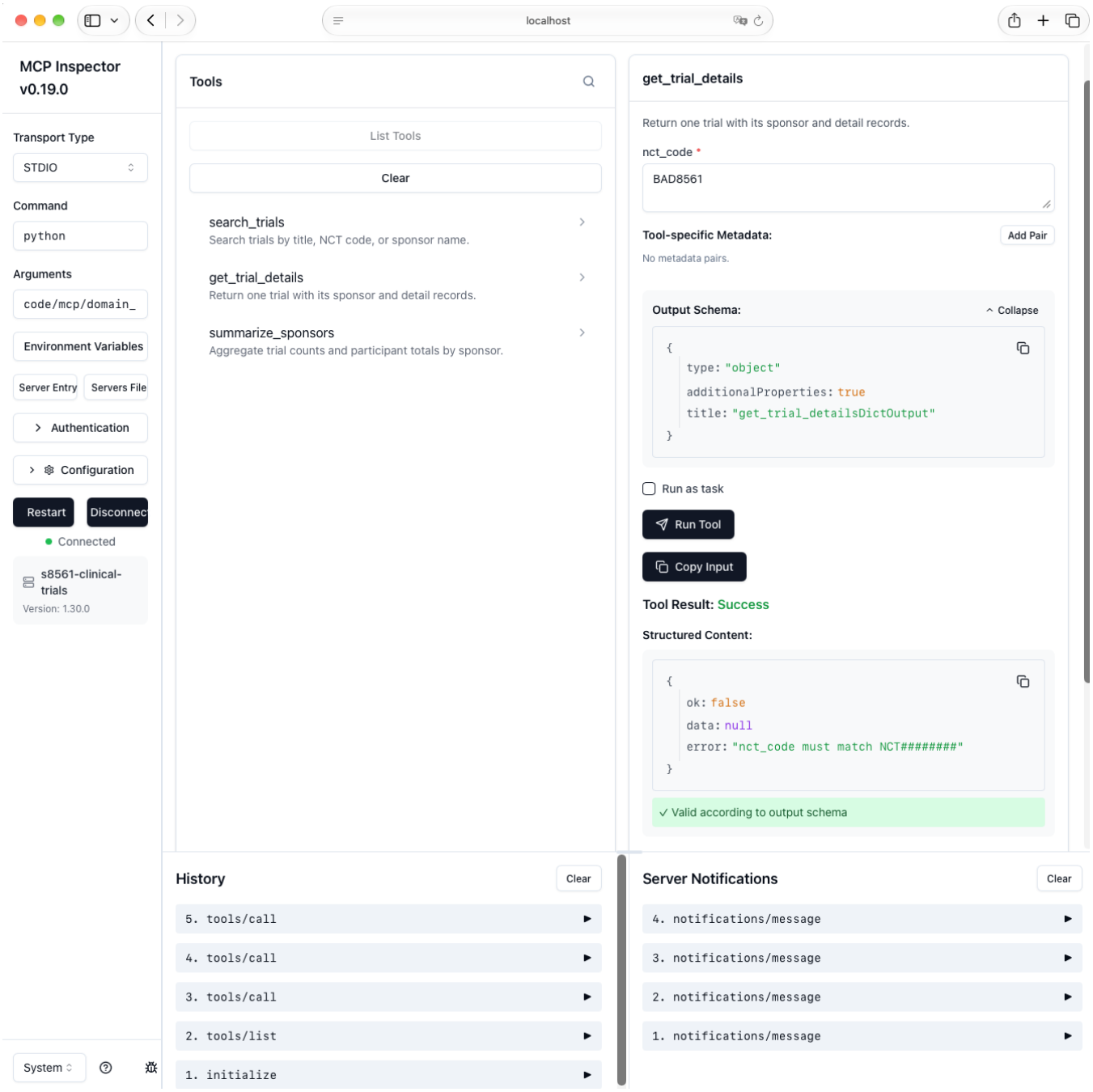


Domain MCP tool - get_trial_details invalid

```
def get_trial_details_data(
    nct_code: str,
    *,
    session_factory: SessionFactory = SessionLocal,
) -> dict[str, Any]:
    """Implementation for one complete trial lookup."""
    if not isinstance(nct_code, str):
        return failure("nct_code must be a string in NCT######## format")
    clean_code = nct_code.strip().upper()
    if not NCT_PATTERN.fullmatch(clean_code):
        return failure("nct_code must match NCT########")
```

```
    def operation() -> dict[str, Any] | None:
        with session_factory() as session:
            statement = (
                select(Trial)
                .options(
```

MCP Inspector - get_trial_details invalid



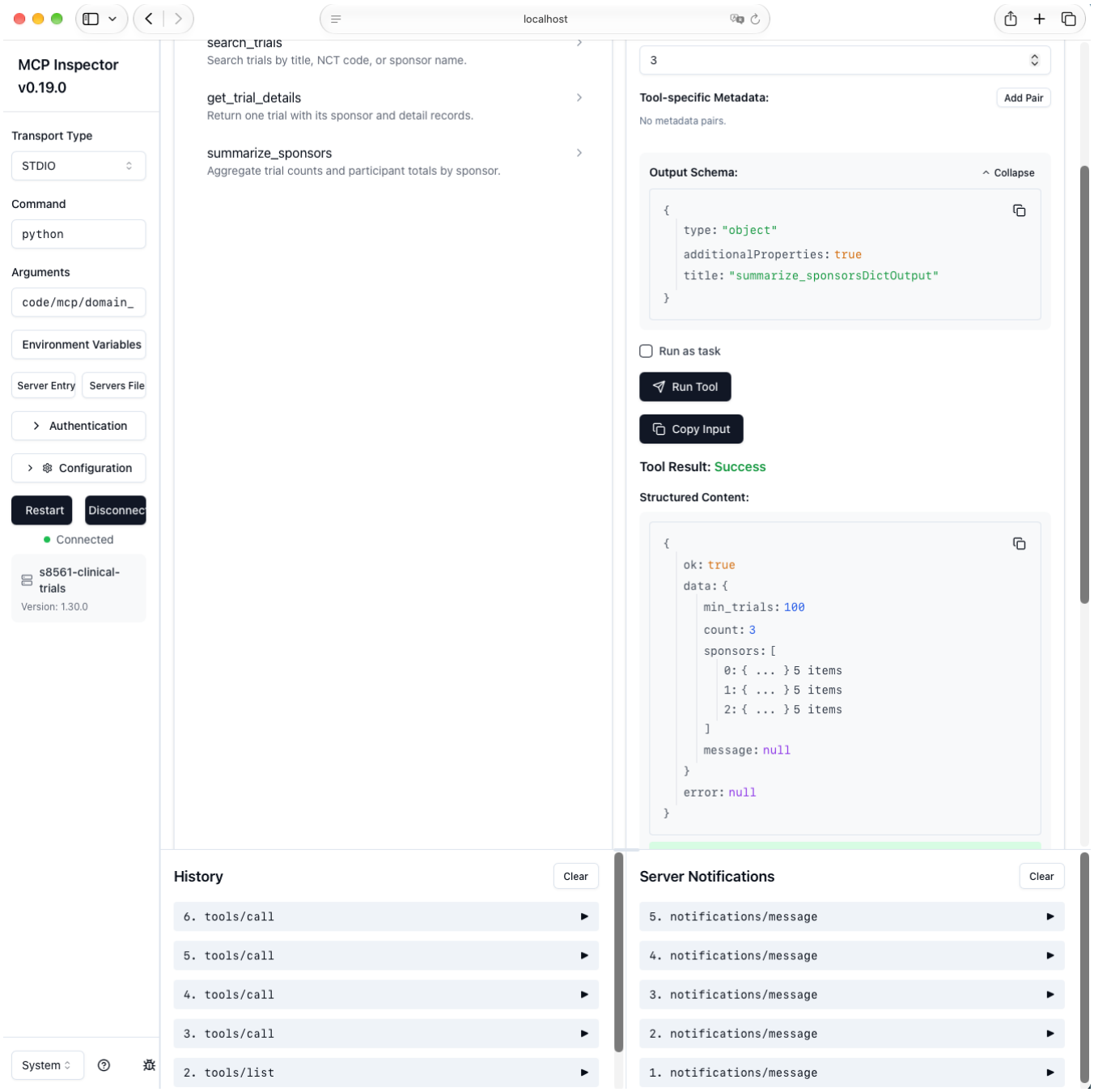
Domain MCP tool - summarize_sponsors success

```
def summarize_sponsors_data(
    min_trials: int = 1,
    limit: int = 10,
    *,
    session_factory: SessionFactory = SessionLocal,
) -> dict[str, Any]:
    """Implementation for aggregate sponsor statistics."""
    if (
        isinstance(min_trials, bool)
        or not isinstance(min_trials, int)
        or not 1 <= min_trials <= 5000
```

```
    ):
        return failure("min_trials must be an integer between 1 and 5000")
    if isinstance(limit, bool) or not isinstance(limit, int) or not 1 <= limit <= 50:
        return failure("limit must be an integer between 1 and 50")

    trial_count = func.count(Trial.id)
```

MCP Inspector - summarize_sponsors success



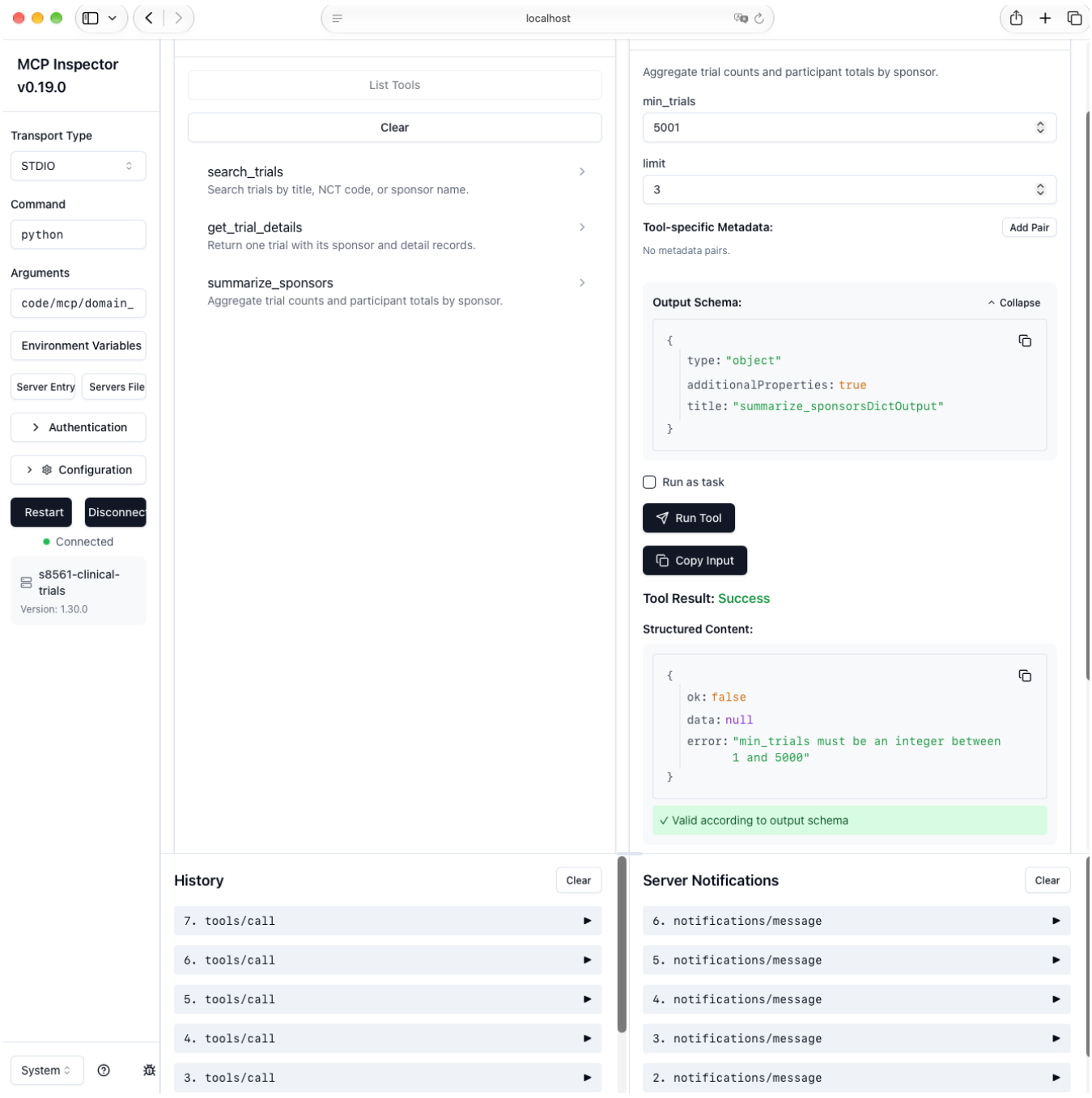
Domain MCP tool - summarize_sponsors invalid

```
def summarize_sponsors_data(
    min_trials: int = 1,
    limit: int = 10,
    *,
    session_factory: SessionFactory = SessionLocal,
) -> dict[str, Any]:
    """Implementation for aggregate sponsor statistics."""
    if (
        isinstance(min_trials, bool)
        or not isinstance(min_trials, int)
```

```
        or not 1 <= min_trials <= 5000
    ):
        return failure("min_trials must be an integer between 1 and 5000")
    if isinstance(limit, bool) or not isinstance(limit, int) or not 1 <= limit <= 50:
        return failure("limit must be an integer between 1 and 50")

    trial_count = func.count(Trial.id)
```

MCP Inspector - summarize_sponsors invalid



Part 3 Tool Contracts Under Stress

Part 3 Tool Contracts Under Stress

All three domain tools return the shared envelope {ok, data, error}. On success, ok is true and error is null. On rejection, ok is false, data is null, and error contains a short explanation.

`search_trials`

Expected input schema:

```
{"query": "string", "limit": "integer from 1 through 25"}
```

Rejected Part 2B input:

```
{"query": "Seeded Clinical Trial 1", "limit": 26}
```

Returned error:

```
{"ok": false, "data": null, "error": "limit must be an integer between 1 and 25"}
```

The call was rejected because 26 exceeds the bounded search-result limit of 25.

`get_trial_details`

Expected input schema:

```
{"nct_code": "string matching NCT########"}
```

Rejected Part 2B input:

```
{"nct_code": "BAD8561"}
```

Returned error:

```
{"ok": false, "data": null, "error": "nct_code must match NCT########"}
```

The call was rejected because the value does not use the required NCT prefix followed by exactly eight digits.

`summarize_sponsors`

Expected input schema:

```
{"min_trials": "integer from 1 through 5000", "limit": "integer from 1 through 50"}
```

Rejected Part 2B input:

```
{"min_trials": 5001, "limit": 3}
```

Returned error:

```
{"ok": false, "data": null, "error": "min_trials must be an integer between 1 and 5000"}
```

The call was rejected because the requested minimum exceeds the 5,000 trials in the seeded domain database.

Bounded timeout and exponential backoff

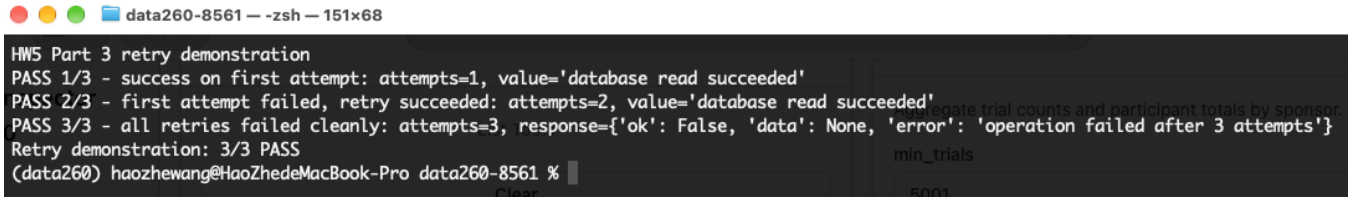
```
def run_with_retry(
    operation: Callable[[], T],
    *,
    timeout_seconds: float = 2.0,
    max_attempts: int = 3,
    base_delay_seconds: float = 0.05,
    max_delay_seconds: float = 0.20,
    retryable_exceptions: tuple[type[BaseException], ...] = (
```

```
        TransientOperationError,
    ),
    should_fail: Callable[[int], bool] | None = None,
    on_retry: Callable[[int, BaseException, float], None] | None = None,
) -> RetryOutcome[T]:
    """Run an operation with a timeout and bounded exponential backoff."""
    if timeout_seconds <= 0:
        raise ValueError("timeout_seconds must be positive")
    if max_attempts < 1:
        raise ValueError("max_attempts must be at least 1")
    if base_delay_seconds < 0 or max_delay_seconds < 0:
        raise ValueError("backoff delays cannot be negative")

    started = time.perf_counter()
    caught_exceptions = retryable_exceptions + (OperationTimeout,)

    for attempt in range(1, max_attempts + 1):
        try:
            if should_fail is not None and should_fail(attempt):
                raise TransientOperationError(
                    f"simulated transient failure on attempt {attempt}"
                )
            value = _call_with_timeout(operation, timeout_seconds)
            latency_ms = (time.perf_counter() - started) * 1000.0
            return RetryOutcome(value, attempt, latency_ms)
        except caught_exceptions as error:
```

Retry policy demonstration



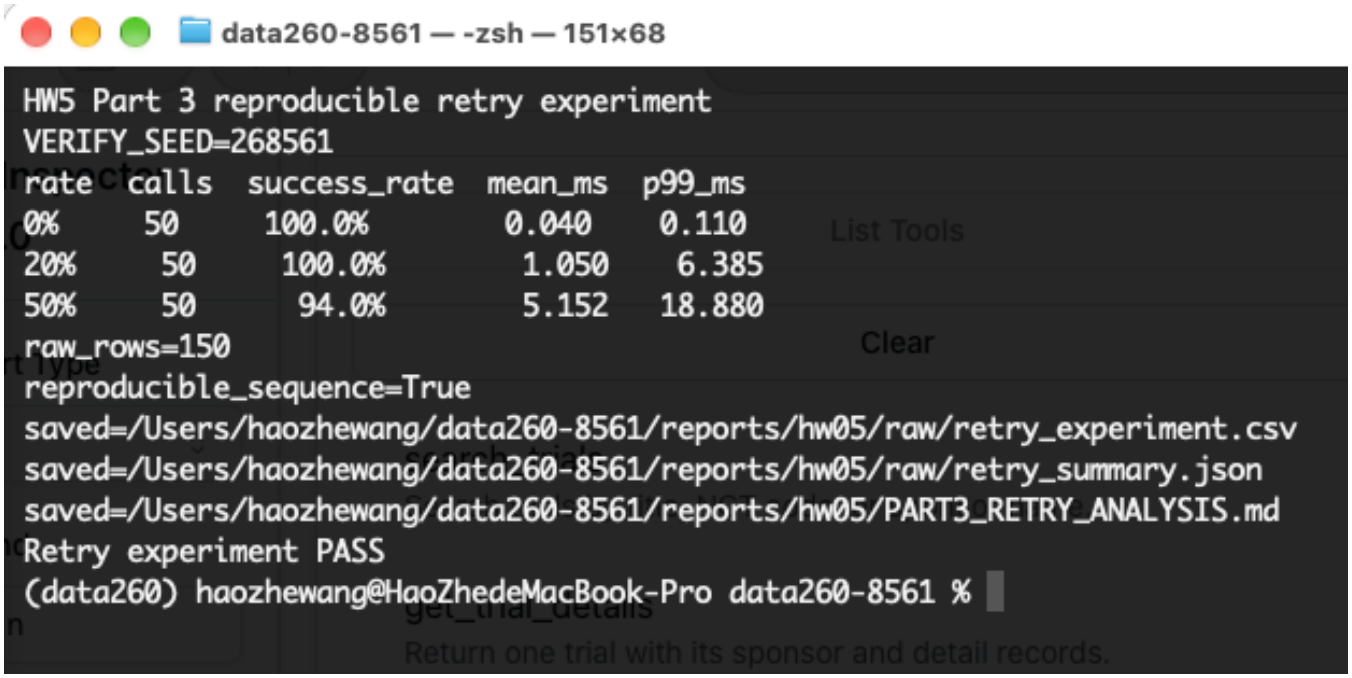
Part 3 Retry Policy Analysis

VERIFY_SEED = 268561. Each failure rate used 50 calls, for 150 calls total. The policy used 3 maximum attempts, a 0.5-second timeout, and bounded exponential backoff from 5 ms to 20 ms.

The policy is suitable for an interactive assistant because three bounded attempts recover most transient failures while keeping worst-case waiting short. The 50% condition shows the tradeoff: retries improve reliability but increase tail latency and still cannot guarantee success.

For batch processing, I would allow more attempts and longer timeouts because throughput and eventual completion matter more than immediate feedback. I would also increase the maximum backoff and add jitter so many workers do not retry at the same time.

Seeded 150 call experiment



Part 4 Safe Tool Entry Point and Offline Tests

Single JSON string tool boundary

```
def execute_tool(
    name: str,
    inputs: Mapping[str, Any],
    *,
    handlers: Mapping[str, ToolHandler] | None = None,
) -> str:
    """Dispatch one domain tool and always return the shared envelope as JSON."""
    registry = DEFAULT_HANDLERS if handlers is None else handlers

    if not isinstance(name, str) or not name.strip():
        return _json_response(failure("tool name must be a non-empty string"))
    clean_name = name.strip()

    if not isinstance(inputs, Mapping):
        return _json_response(failure("inputs must be a JSON object"))
    if clean_name not in registry:
        return _json_response(failure(f"unknown tool: {clean_name}"))

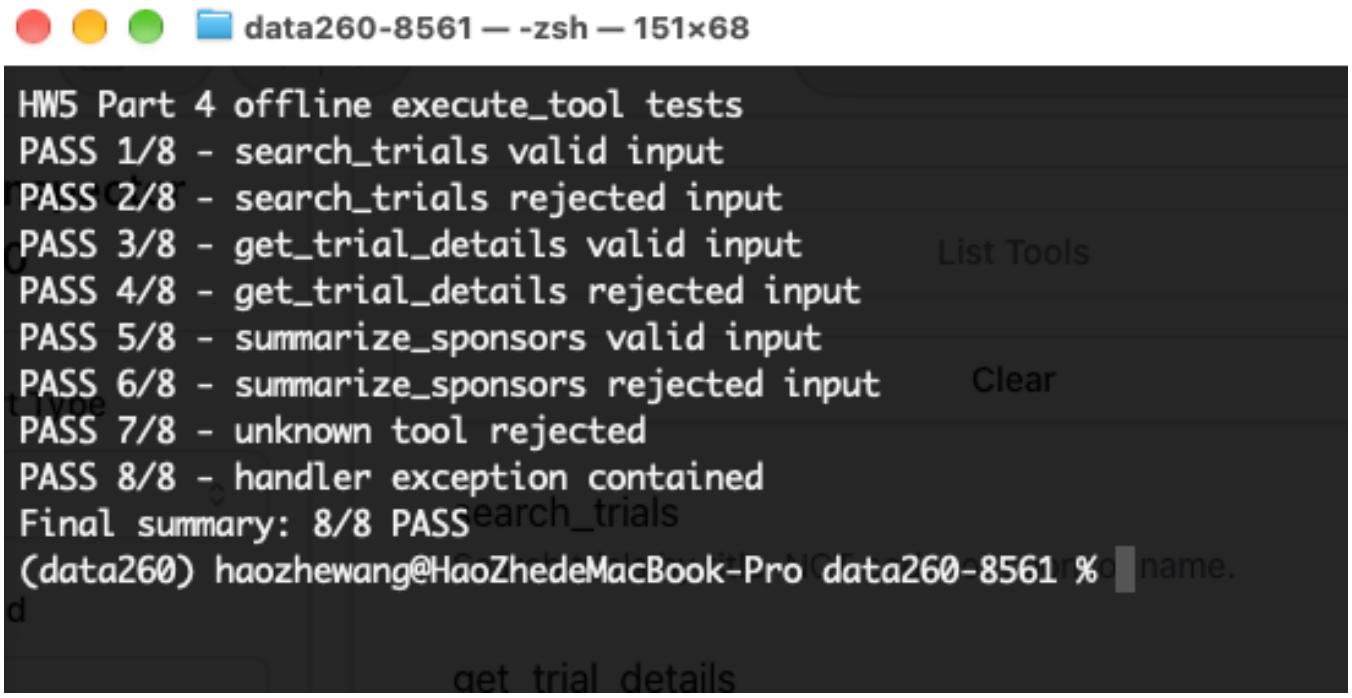
    if clean_name == "summarize_sponsors":
        min_trials = inputs.get("min_trials", 1)
        if (
            isinstance(min_trials, int)
            and not isinstance(min_trials, bool)
            and min_trials < MIN_SAFE_AGGREGATE_TRIALS
        ):
            return _json_response(
                failure(
                    "safety rule: sponsor aggregates require min_trials "
                    f"of at least {MIN_SAFE_AGGREGATE_TRIALS}"
                )
            )

    try:
        result = registry[clean_name](**dict(inputs))
    except TypeError as error:
        return _json_response(failure(f"invalid inputs for {clean_name}: {error}"))
    except Exception as error:  # noqa: BLE001 - this boundary must never crash callers
        return _json_response(
            failure(
                f"{clean_name} failed cleanly: {type(error).__name__}"
            )
        )
```

Part 4 Offline Tool-Entry Tests

code/mcp/tool_executor.py defines the single execute_tool(name, inputs) entry point used by the later agent. It dispatches only the three allowlisted domain tools and serializes the shared {ok, data, error} envelope as a JSON string. Unknown tools, malformed inputs, handler exceptions, and invalid handler return values are converted to clean error envelopes rather than escaping as crashes. code/mcp/test_tool_executor.py creates a temporary SQLite database and injects fixture-backed handlers. It does not connect to MySQL and does not invoke an LLM. The runner covers valid and rejected inputs for each of the three tools, plus an unknown tool and an injected internal exception, for eight tests total.

Offline test runner



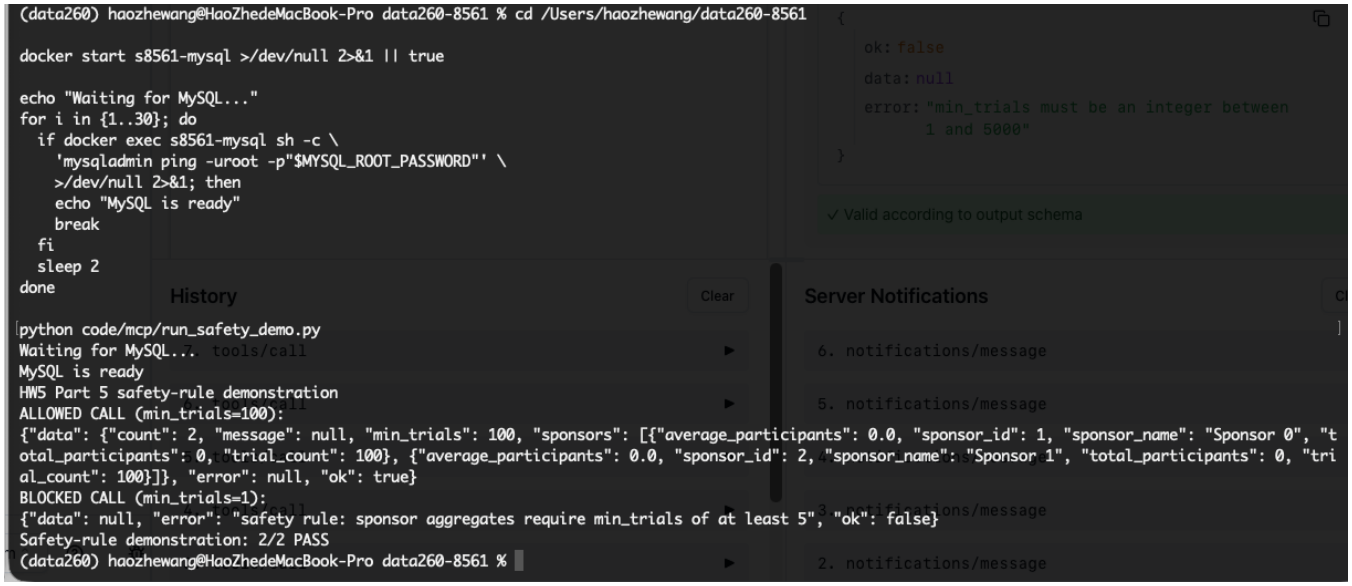
Part 5 Agent Loop Safety Metrics and Reflection

Domain safety rule

```
    if clean_name == "summarize_sponsors":
        min_trials = inputs.get("min_trials", 1)
        if (
            isinstance(min_trials, int)
            and not isinstance(min_trials, bool)
            and min_trials < MIN_SAFE_AGGREGATE_TRIALS
        ):
            return _json_response(
                failure(
                    "safety rule: sponsor aggregates require min_trials "
                    f"of at least {MIN_SAFE_AGGREGATE_TRIALS}"
                )
            )

    try:
        result = registry[clean_name](**dict(inputs))
    except TypeError as error:
        return _json_response(failure(f"invalid inputs for {clean_name}: {error}"))
    except Exception as error:  # noqa: BLE001 - this boundary must never crash callers
        return _json_response(
            failure(
                f"{clean_name} failed cleanly: {type(error).__name__}"
            )
        )
```

Allowed and blocked safety calls



Bounded Ollama agent loop

```
def run_agent(
    user_input: str,
    *,
    model: Any | None = None,
    max_steps: int = 4,
    executor: ToolExecutor = execute_tool,
    log_path: Path = DEFAULT_LOG_PATH,
) -> dict[str, Any]:
    """Run a bounded tool loop and record every action and stop reason."""
    if not isinstance(user_input, str) or not user_input.strip():
        raise ValueError("user_input must be a non-empty string")
    if max_steps < 1:
        raise ValueError("max_steps must be at least 1")

    active_model = model if model is not None else OllamaDecisionModel()
    messages: list[dict[str, str]] = [
        {"role": "user", "content": user_input.strip()}
    ]
    steps: list[dict[str, Any]] = []
```

```
tool_call_count = 0
stop_reason = "max_steps"
final_response = "Stopped after reaching the maximum number of steps."

for step_number in range(1, max_steps + 1):
    try:
        action = active_model.next_action(messages, TOOL_DESCRIPTIONS)
    except Exception as error:  # noqa: BLE001 - agent boundary must be safe
        action = {
            "type": "model_error",
            "error": f"{type(error).__name__}: {error}",
        }
        steps.append({"step": step_number, "model_action": action})
        stop_reason = "model_error"
        final_response = "The local model failed to produce a valid action."
        break

    step: dict[str, Any] = {
        "step": step_number,
        "model_action": action,
    }

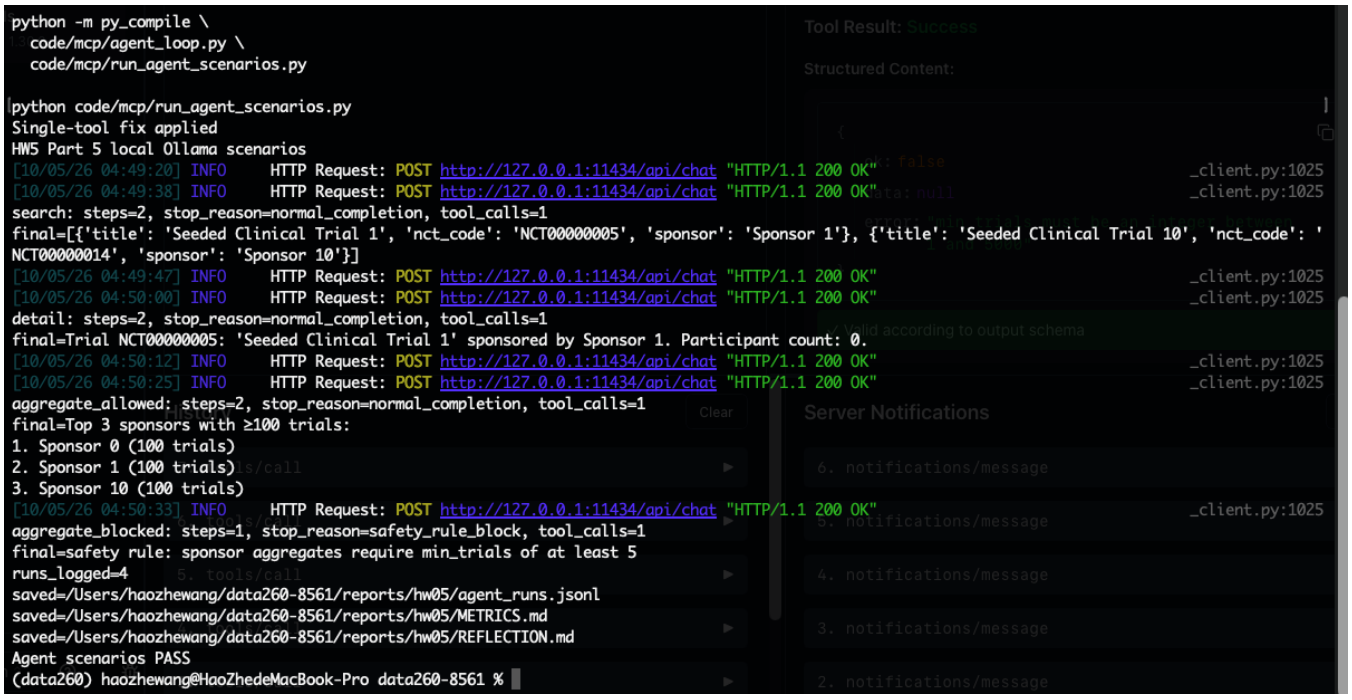
    if action.get("type") == "final":
        final_response = str(action.get("content") or "").strip()
        step["final_response"] = final_response
        steps.append(step)
        stop_reason = "normal_completion"
        break

    name = str(action.get("name") or "")
    inputs = action.get("inputs")
    if not isinstance(inputs, Mapping):
        inputs = {}
    raw_result = executor(name, inputs)
    try:
        result = json.loads(raw_result)
    except json.JSONDecodeError:
```

HW5 Part 5 Agent Metrics

Local model: qwen3:8b.

Four local Ollama scenarios



HW5 Part 5 Reflection

I selected the aggregate_blocked run from agent_runs.jsonl. The user prompt was:

“Your first action must call summarize_sponsors with exactly min_trials 1 and limit 3. Submit those exact values unchanged so the application safety boundary can evaluate them.” The harness first placed this request in the conversation and asked the

local qwen3:8b model for one JSON action. The system prompt listed only the three

approved clinical-trial tools and required the model to obtain database facts through

those tools rather than inventing an answer.

The model returned a structured tool action. The harness recorded that action, checked

the tool name, and sent its input to the single execute_tool boundary. That boundary

applied schema validation and the domain safety policy before any database work. The

safety policy requires Sponsor aggregates to represent at least five trials. This rule

reduces the risk of presenting very small groups as meaningful or privacy-safe

aggregates. The returned {ok, data, error} envelope was added to both the run log and

the model conversation, so every decision remained inspectable.

This run ended with stop reason safety_rule_block after 1 step(s) and

1 tool call(s). The harness stopped at the policy boundary instead of asking

the model to work around the refusal. That behavior is important: the model proposes

actions, but deterministic application code owns authorization and stopping decisions.

The max_steps ceiling supplies a second independent safeguard against looping. This

run demonstrates that logging, validation, policy enforcement, and bounded control flow

work together to make the agent easier to audit than an unrestricted model-to-database

connection.

Word count: 254

AI Use

HW5 AI Use

1. What I used an AI assistant for and what I did myself

I used an AI assistant to help plan the implementation, draft small code changes, suggest test cases, and organize the required evidence. I ran every command locally, inspected the database and API results, exercised the MCP tools in Inspector, captured the screenshots, and verified the final behavior myself.

2. One AI-produced output that was wrong or unsuitable

The first Part 5 agent scenario prompt was unsuitable. In the blocked aggregate scenario, the local model changed min_trials=1 to min_trials=5, so the run ended normally instead of exercising the safety boundary. A later search scenario also tried to call a second, unrelated tool after receiving its first tool result.

3. How I detected or verified the problem

I detected both problems from the structured console output and agent_runs.jsonl. The reported stop reason was normal_completion instead of safety_rule_block, and the tool-call count was larger than expected. I added assertions for the expected stop reason and exactly one tool call per scenario, then reran all four scenarios.

4. What I changed and why it works now

I clarified that exact user-supplied inputs must reach application validation unchanged and that the model must return a final action immediately after one tool result. The scenario runner now fails instead of printing PASS when a stop reason or tool-call count is wrong. The final run recorded three normal completions and one safety-rule block, with exactly one tool call in every scenario.

Reproducibility and Raw Evidence

The repository includes RUN_LOG.txt, verification.json, all 150 fault injection records, machine readable MCP calls, the Part 5 JSONL log, metrics, and the labeled screenshots used in this report.